

Chef Introductory Workshop

Nathen Harvey
nharvey@opscode.com
[@nathenharvey](https://twitter.com/nathenharvey)





- Welcome!
- Did you complete the pre-requisites?
- Nathen Harvey
 - nharvey@opscode.com
 - @nathenharvey

Introductions



- Technical Community Manager
- Co-host Food Fight Show
- @nathenharvey



Introduce yourselves

Objectives and Expectations



- Overview of Chef
- Setup workstation environment
- Building Projects in Chef
- Anatomy of a Chef Run
- Hands on exercises

- This is a half-day workshop, not a comprehensive course.
- We will do some hands on exercises.
- You should get a taste for automating with Chef.

Overview of Chef

What is this thing again?



- Just build it
- Keep notes in `server.txt`
- Move notes to the wiki
- Custom scripts (in scm?!)
- Snapshot & Clone

Chef is an automation platform for developers & systems engineers to continuously define, build, and manage infrastructure.

CHEF USES:

 **Recipes** and  **Cookbooks**
that describe Infrastructure as Code.

Chef enables people to easily build & manage complex & dynamic applications at massive scale

- New model for describing infrastructure that promotes reuse
- Programmatically provision and configure
- Reconstruct business from code repository, data backup, and bare metal resources

“



@VanessaAlvarez1
Vanessa Alvarez

FORRESTER®

#IOF11 #fidelity uses @opscode #chef for developers team, automated asset mgmt platform, time to market was 1300% improvement! #fidelity”

9 Nov via TweetDeck





<http://www.flickr.com/photos/sbh/462754460/>

- Networking
- Files
- Directories
- Symlinks
- Mounts

- Routes
- Users
- Groups
- Tasks
- Packages
- Software
- Services
- Configuration
- Other Stuff







OPSCODE

To Provide a Service



<http://www.flickr.com/photos/28309157@N08/3743455858/>





Application Server



Application Server



Application Database



Application Server



Application Databases



Application Servers



Application Databases



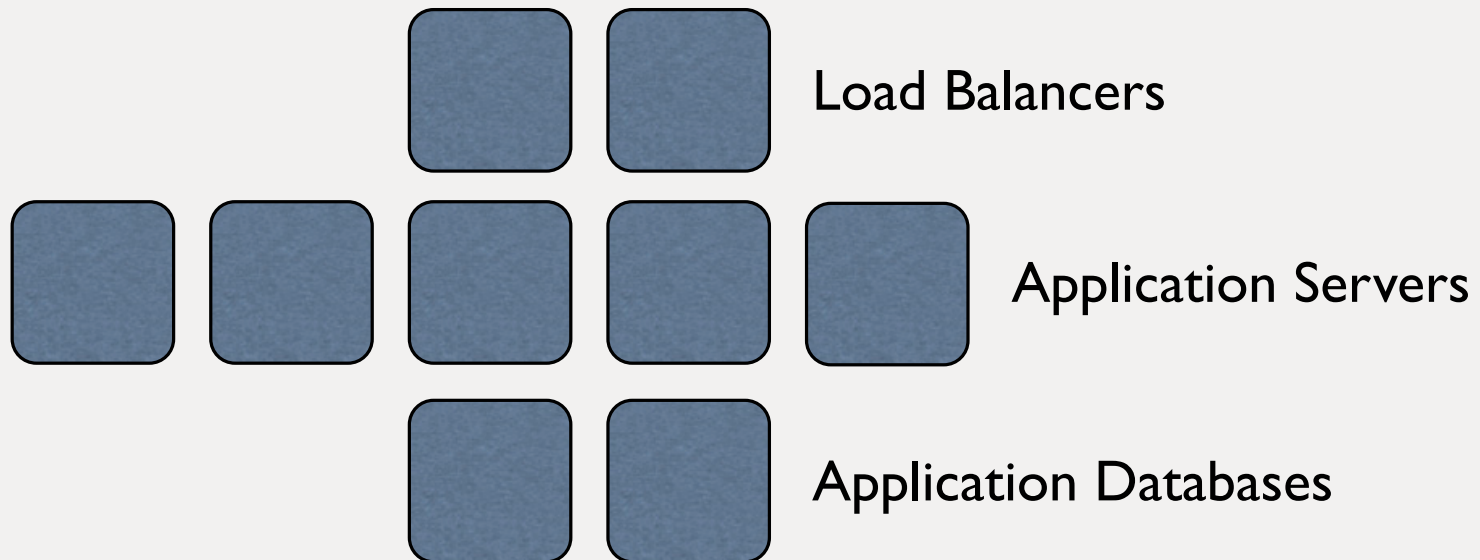
Load Balancer

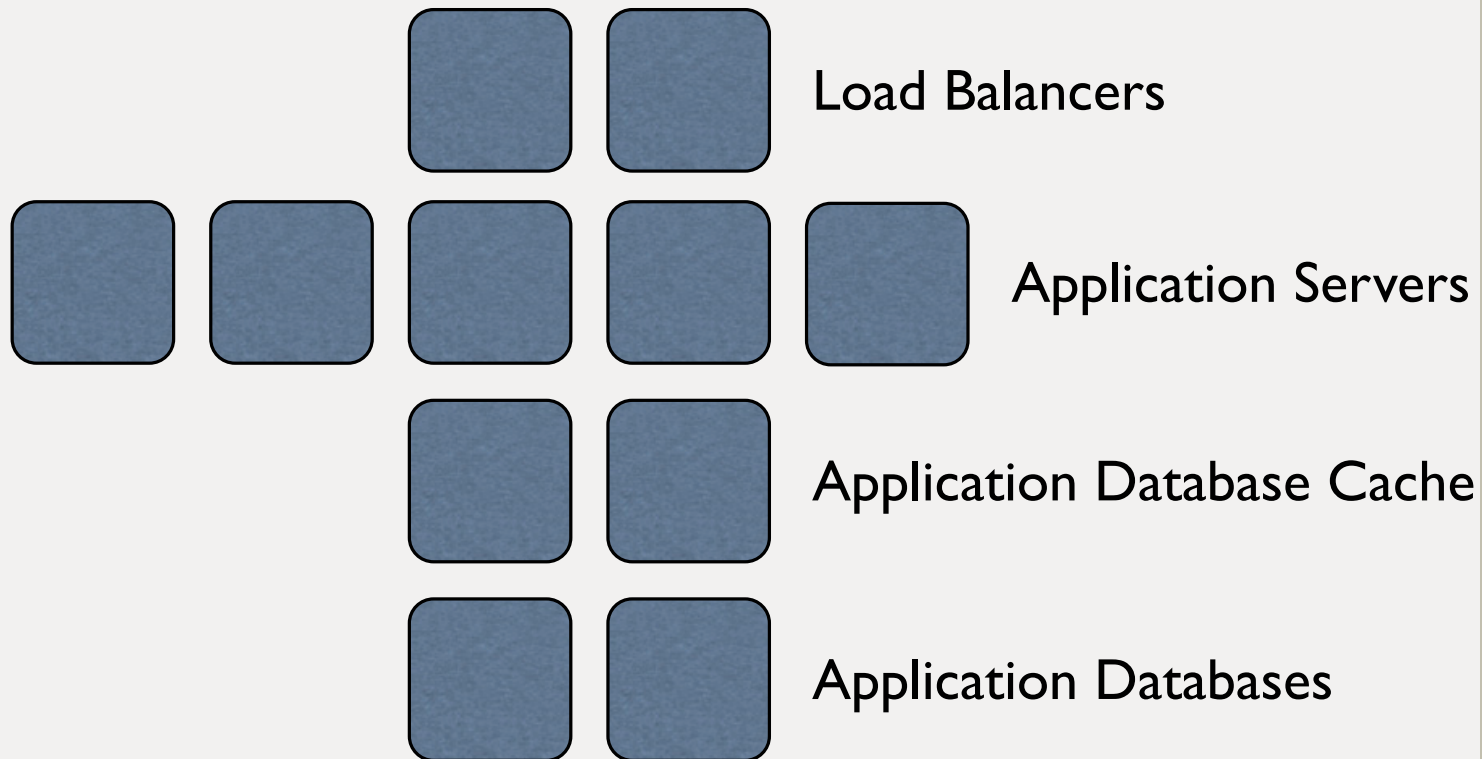


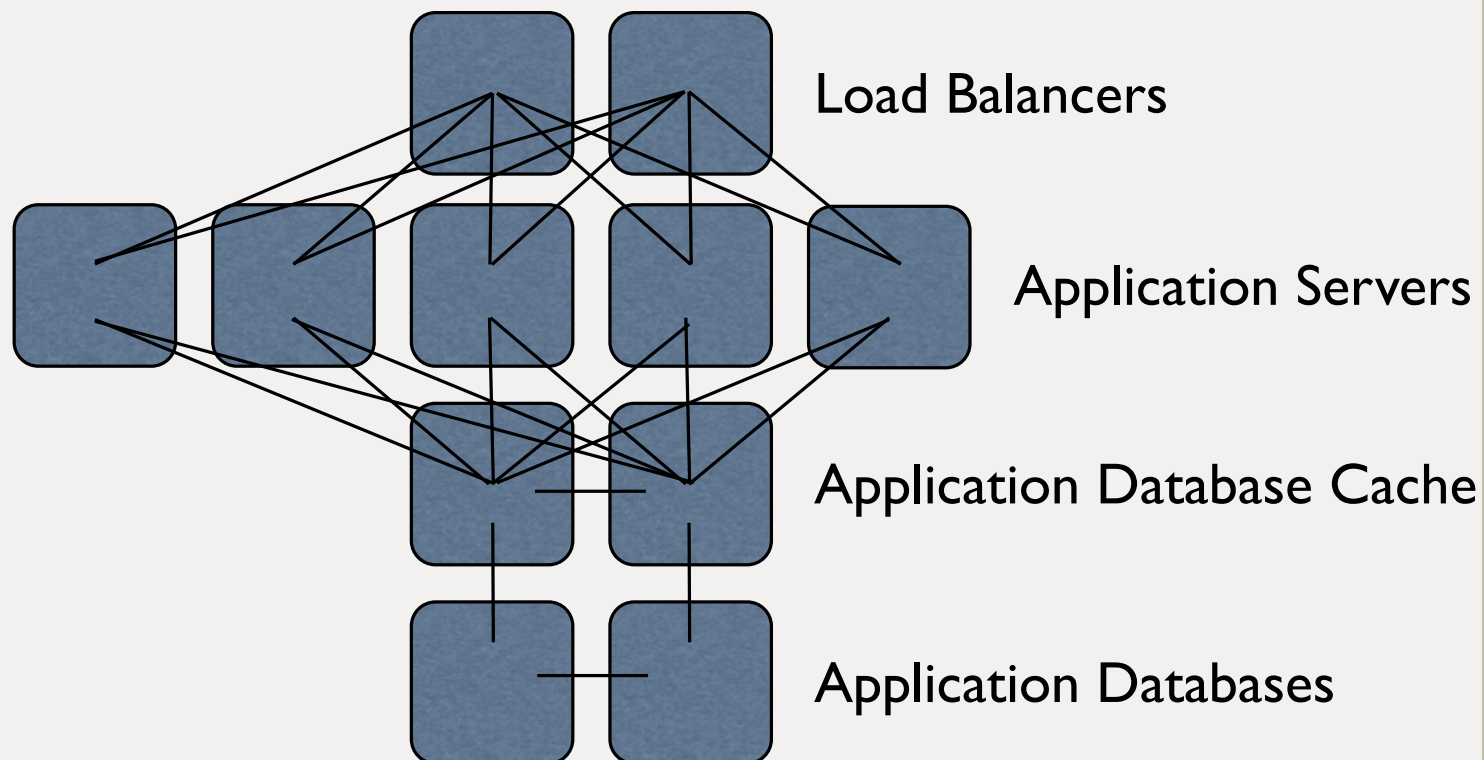
Application Servers

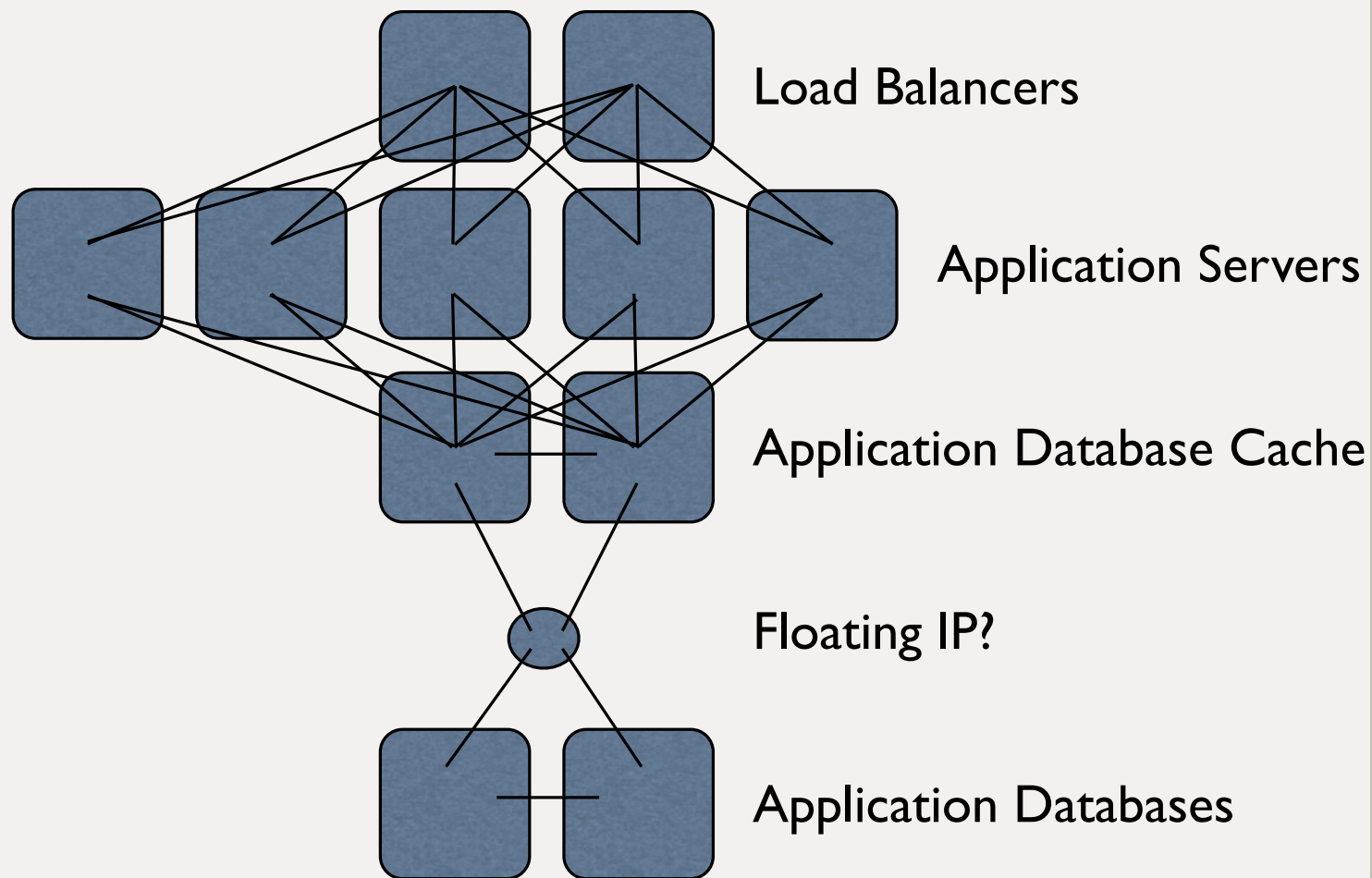


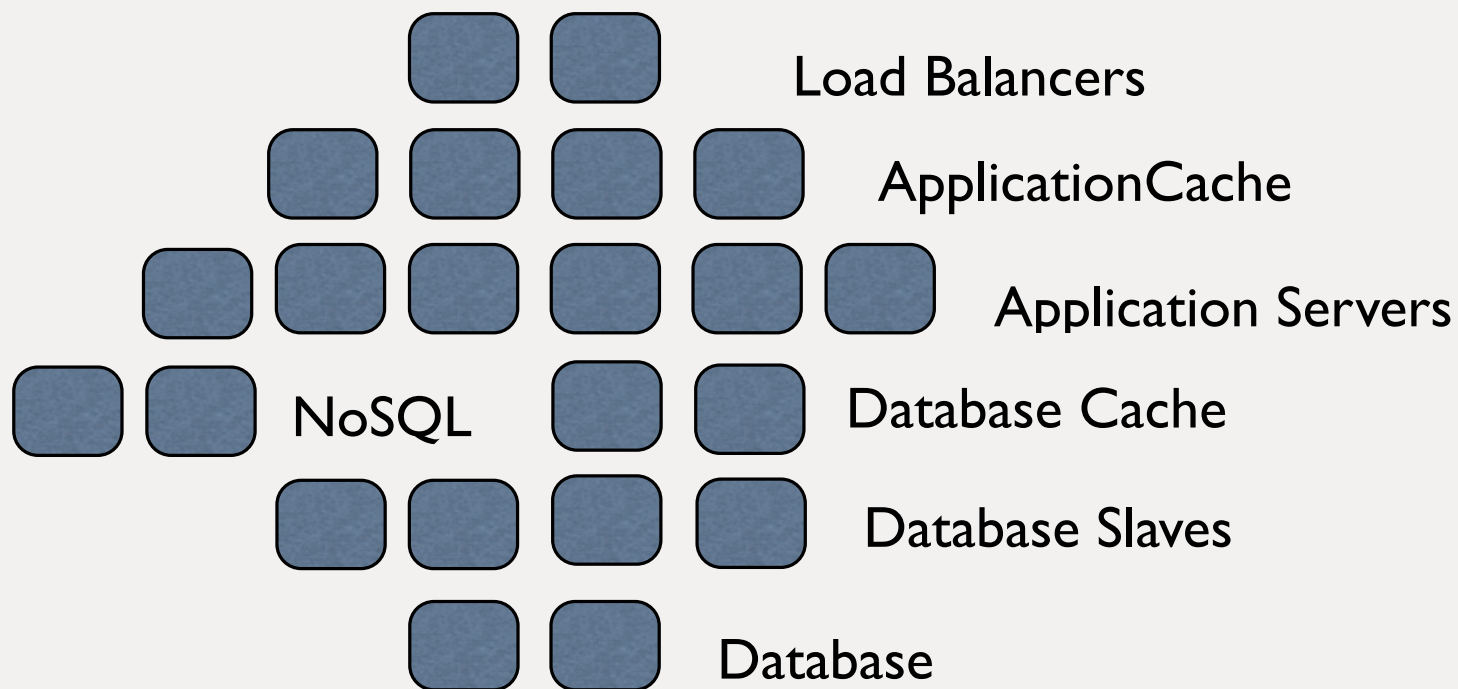
Application Databases

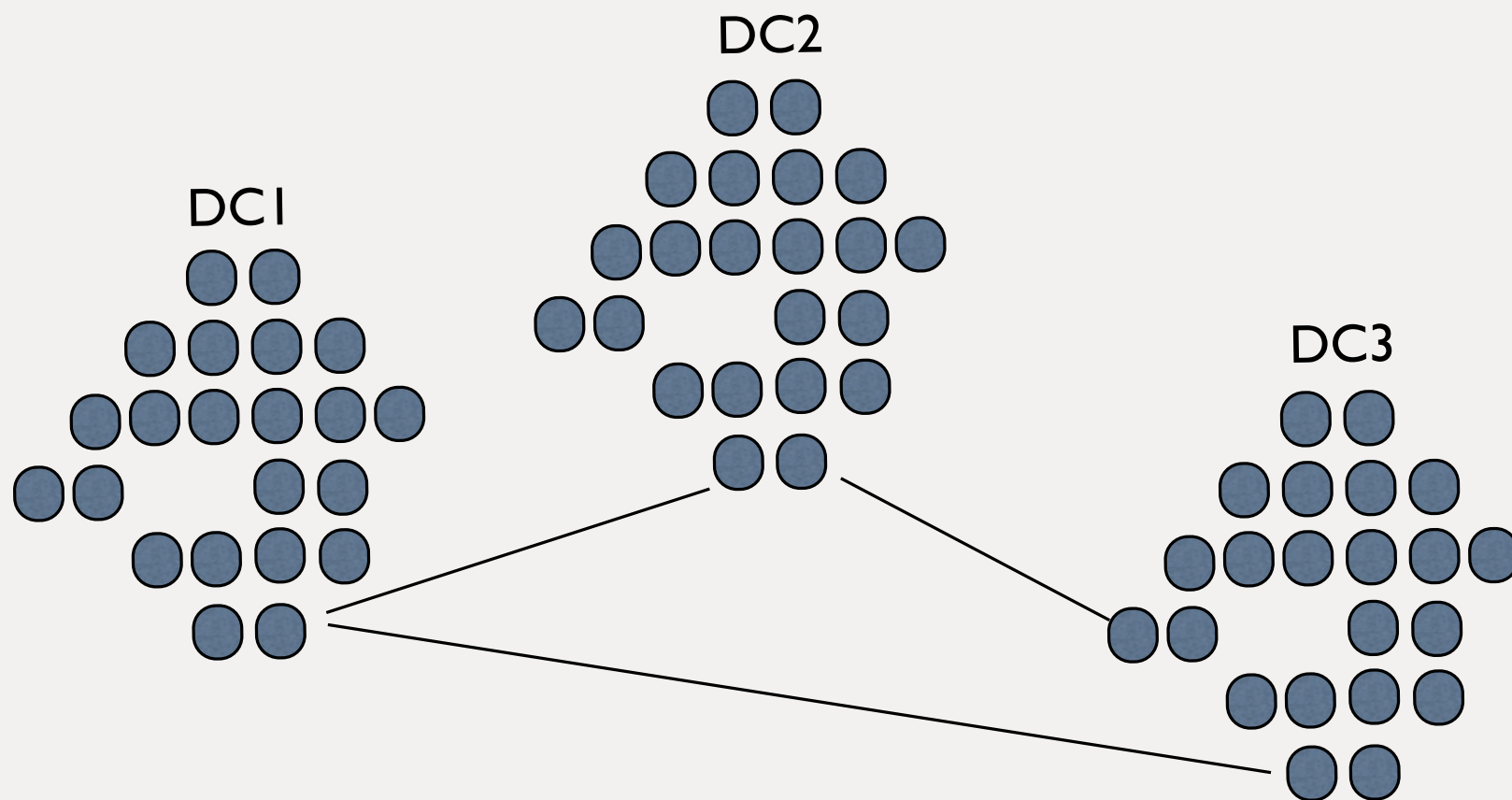


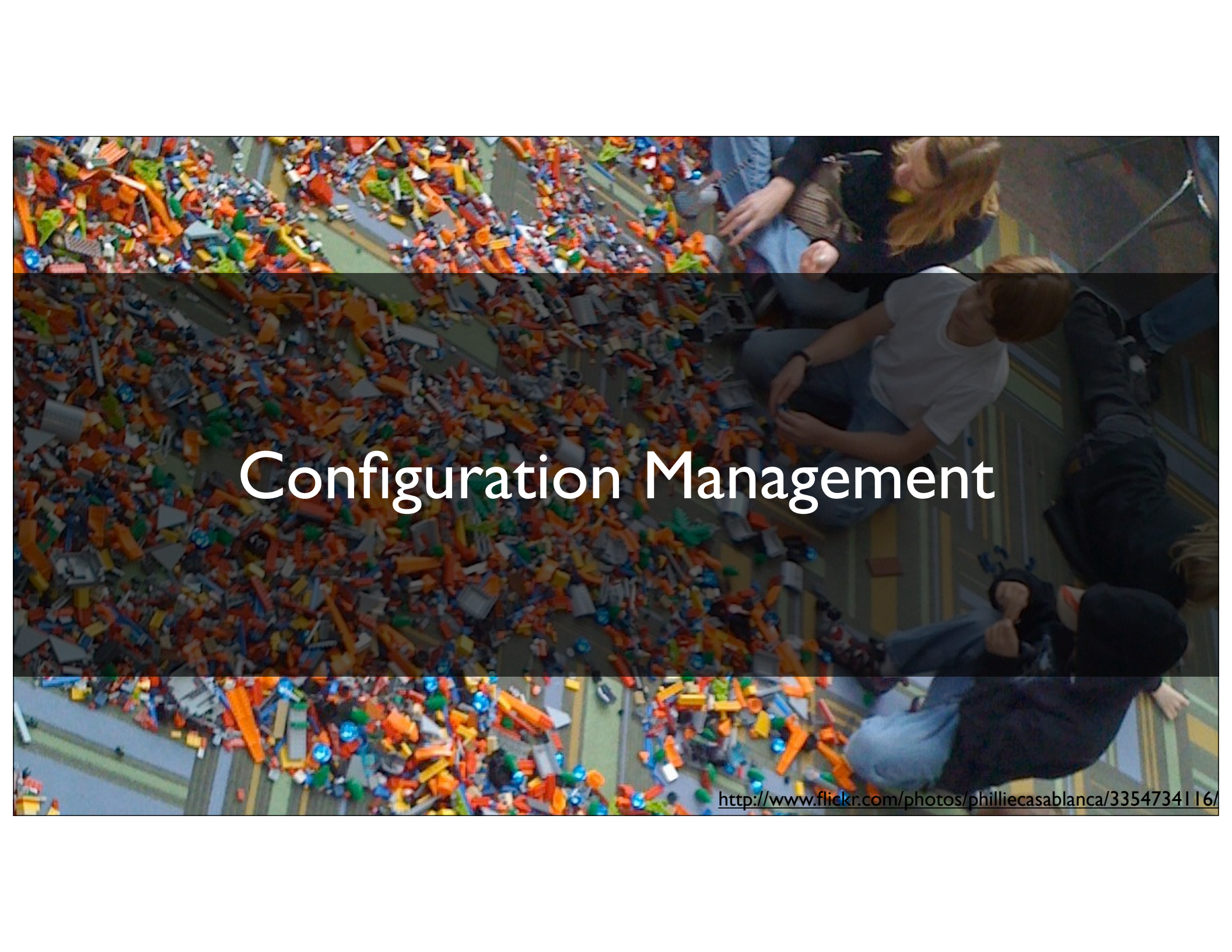












Configuration Management

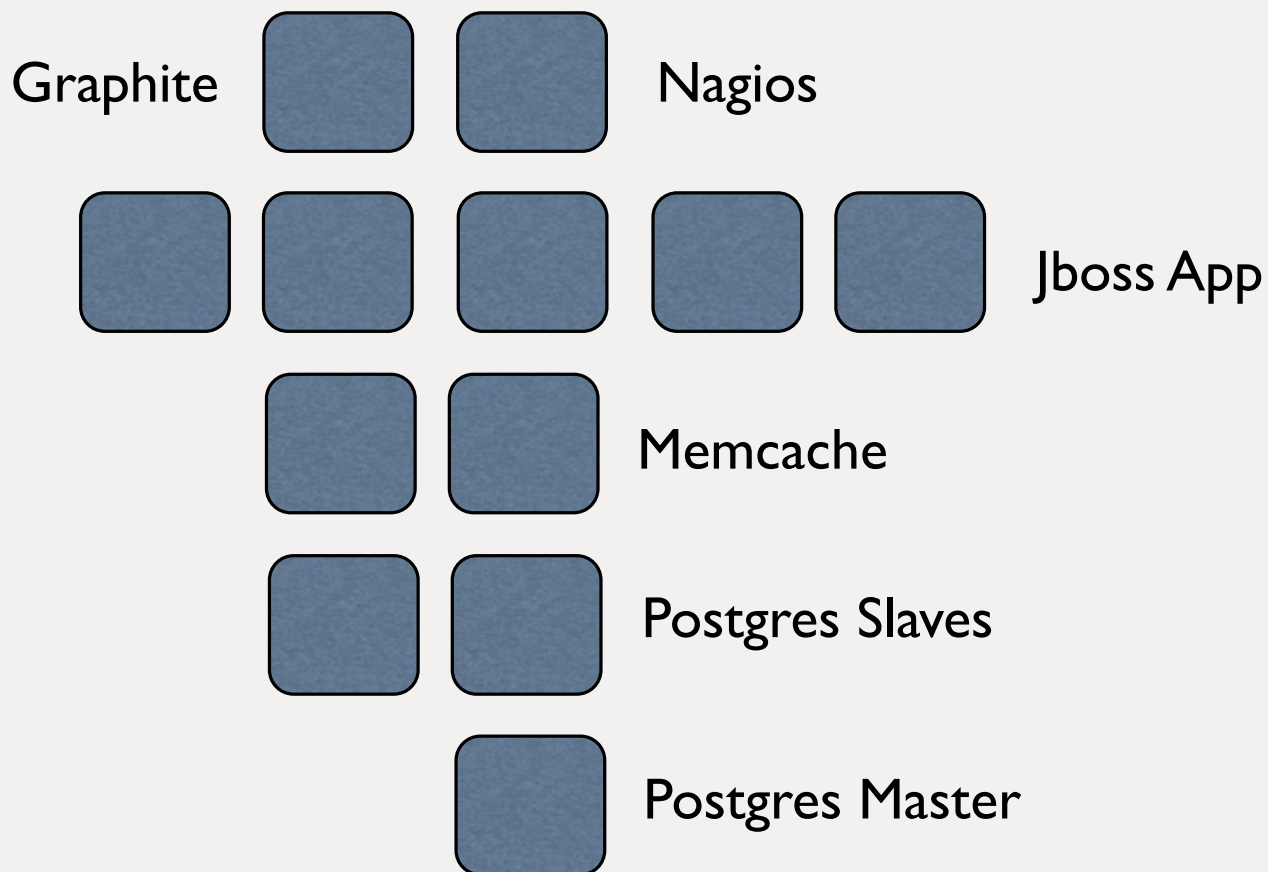
<http://www.flickr.com/photos/philliecasablanca/3354734116/>

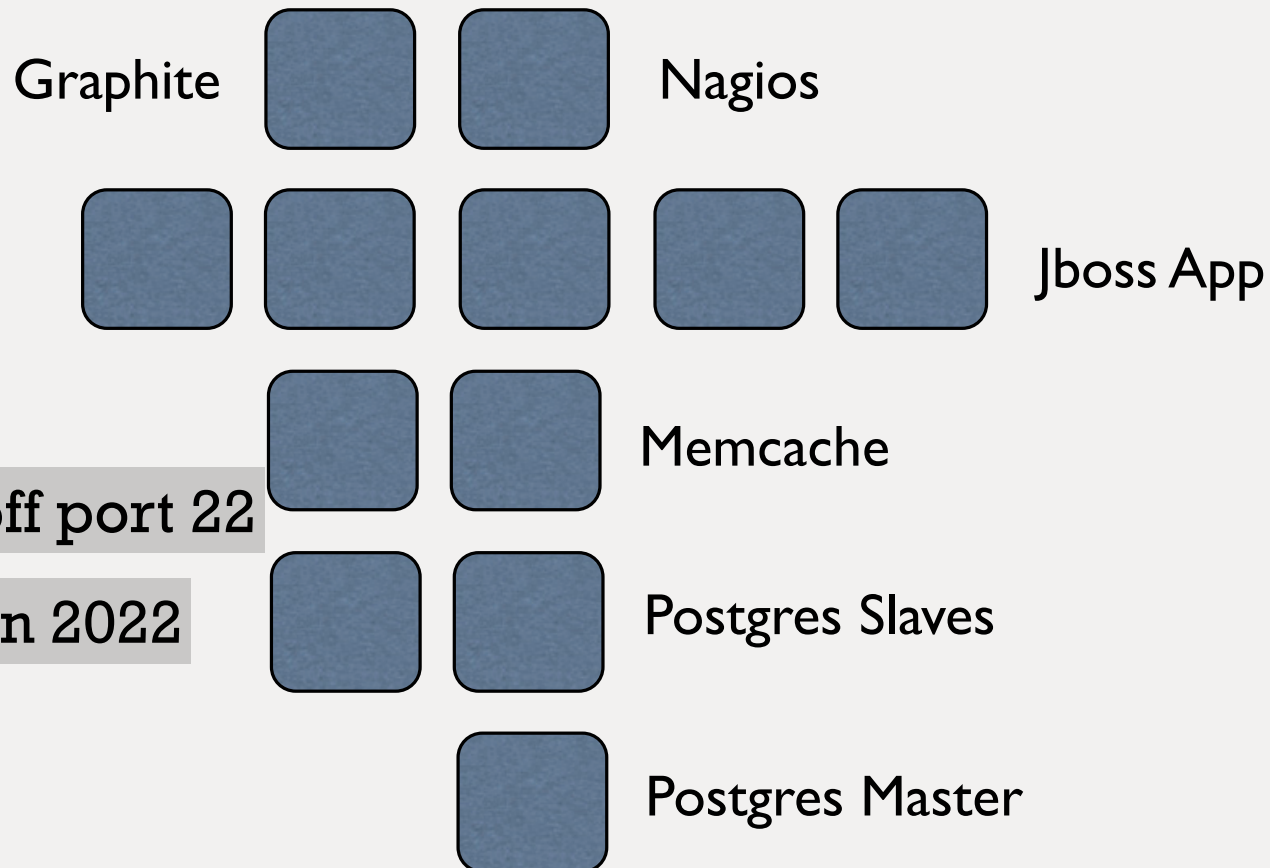
Golden Images are not the answer

- Gold is heavy
- Hard to transport
- Hard to mold
- Easy to lose configuration detail

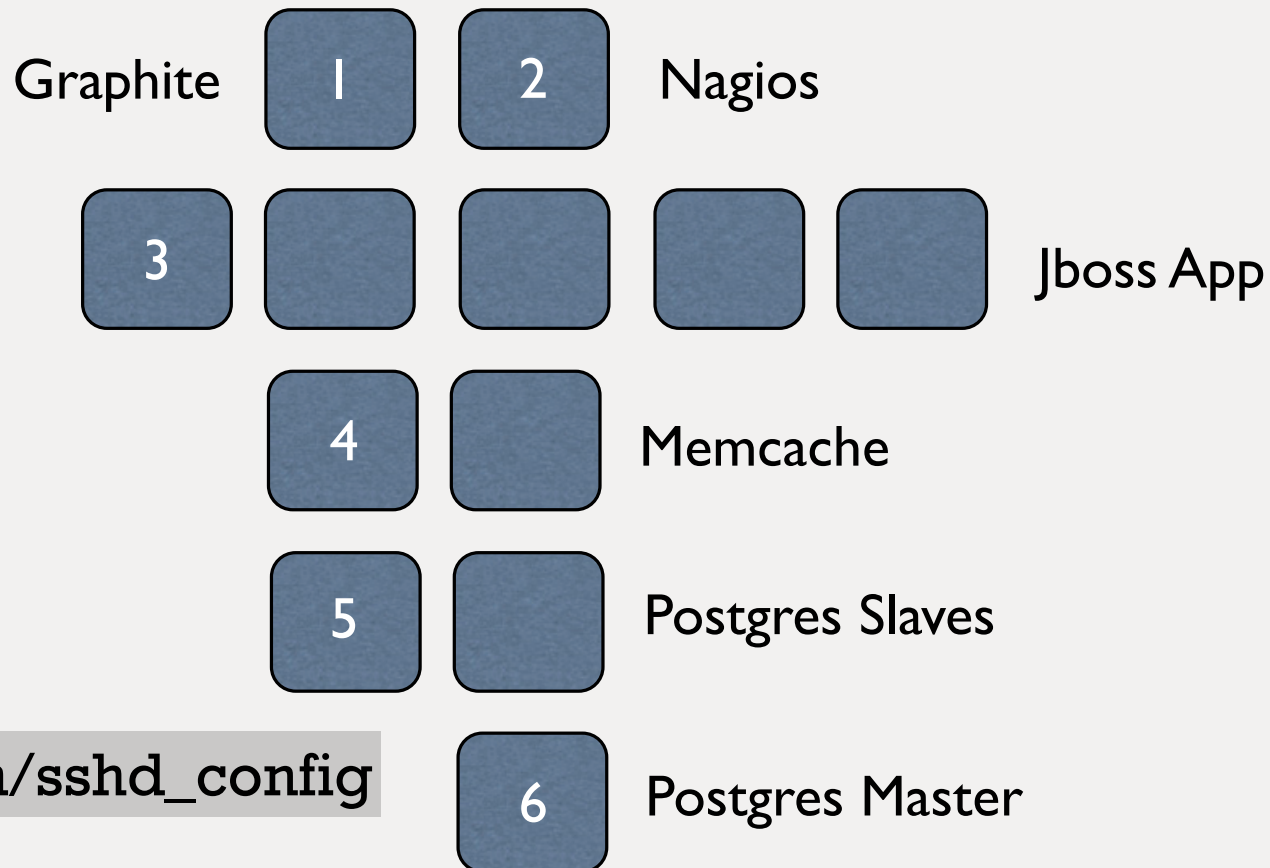
<http://www.flickr.com/photos/garysoup/2977173063/>



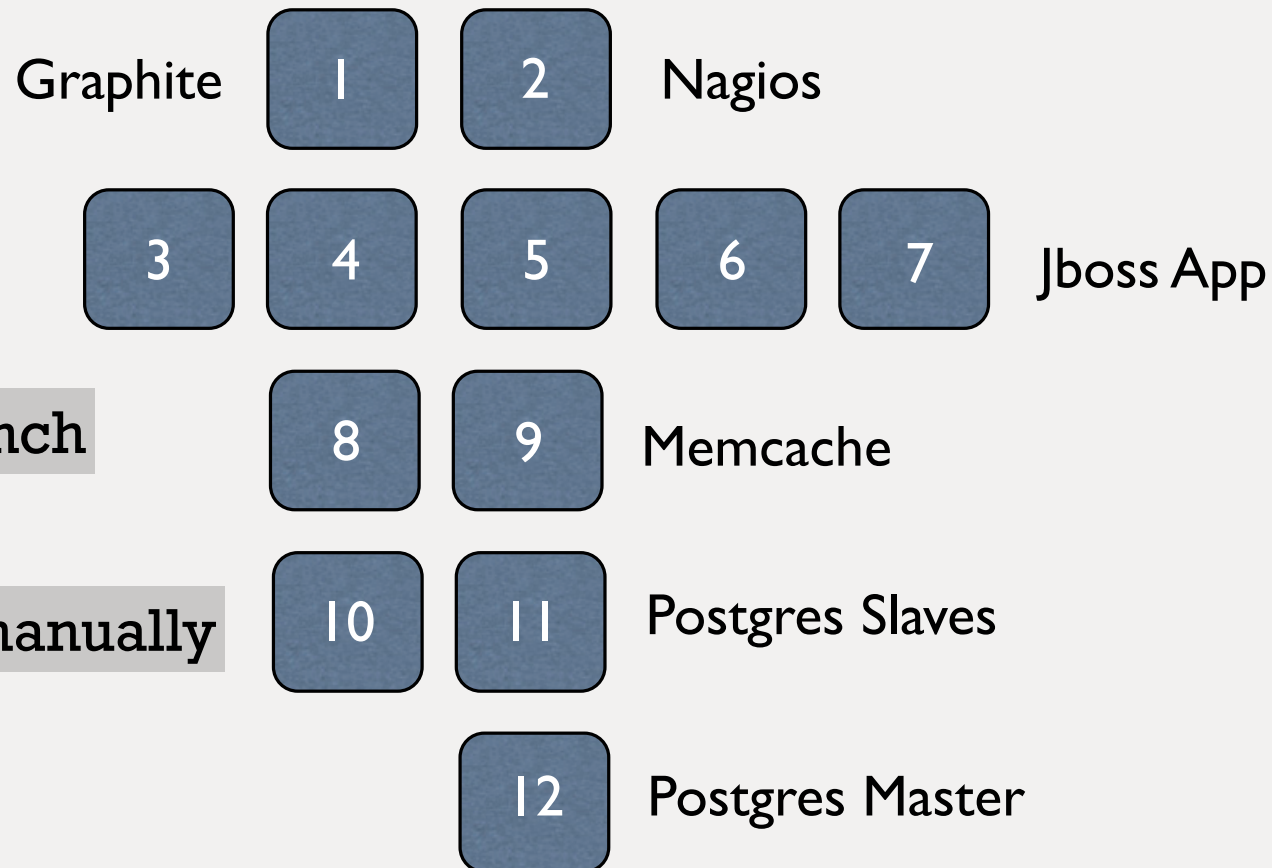




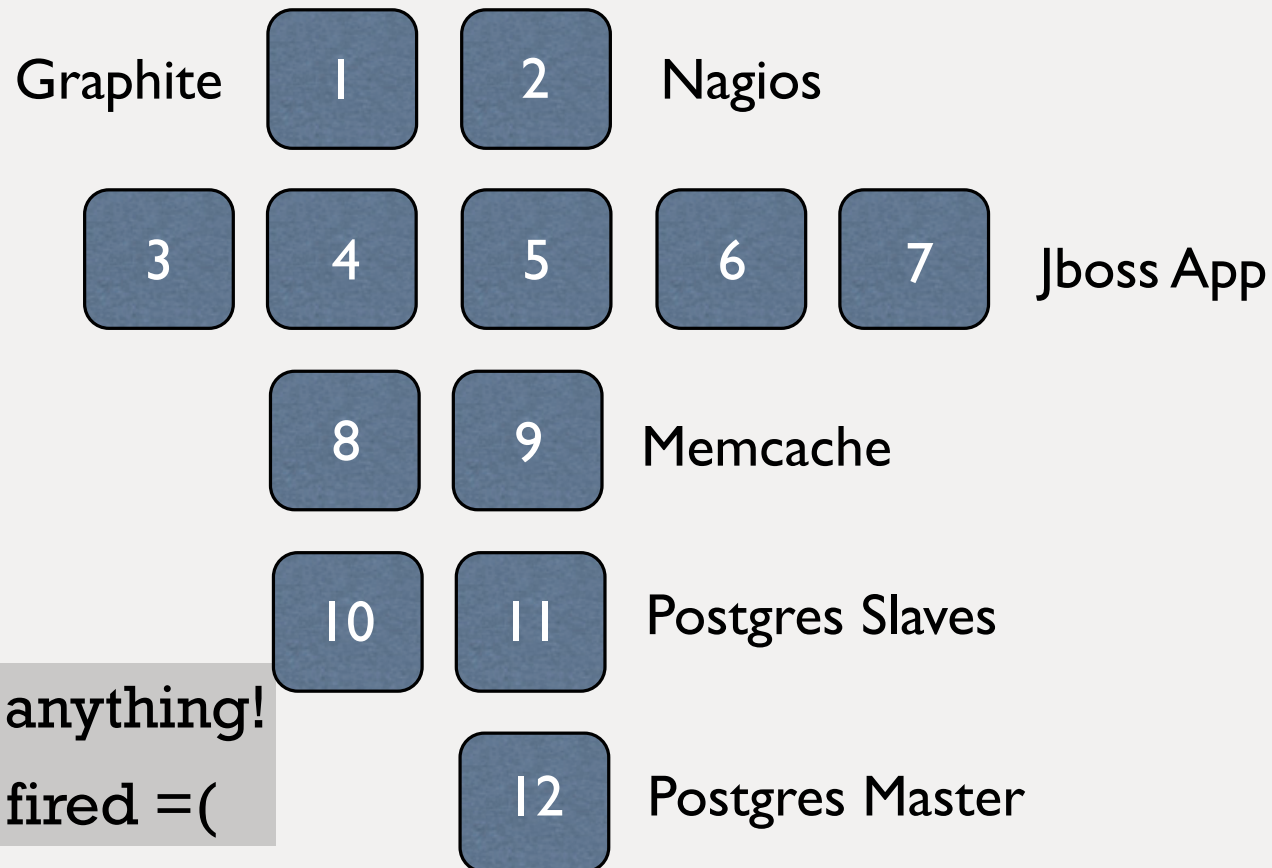
- Move SSH off port 22
- Lets put it on 2022



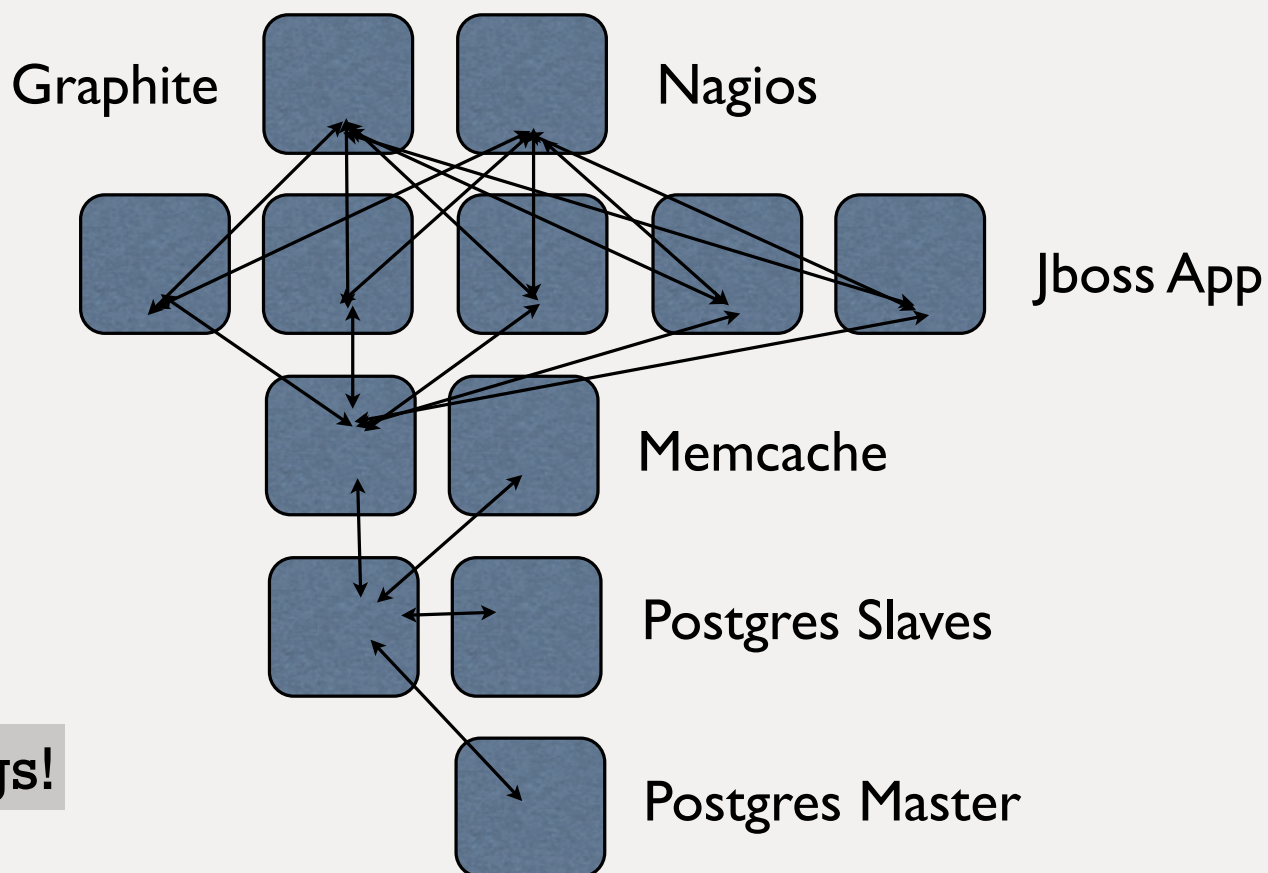
- `edit /etc/ssh/sshd_config`



- Delete, launch
- Repeat
- Typically manually



- Don't break anything!
- Bob just got fired =(



● Invalid configs!

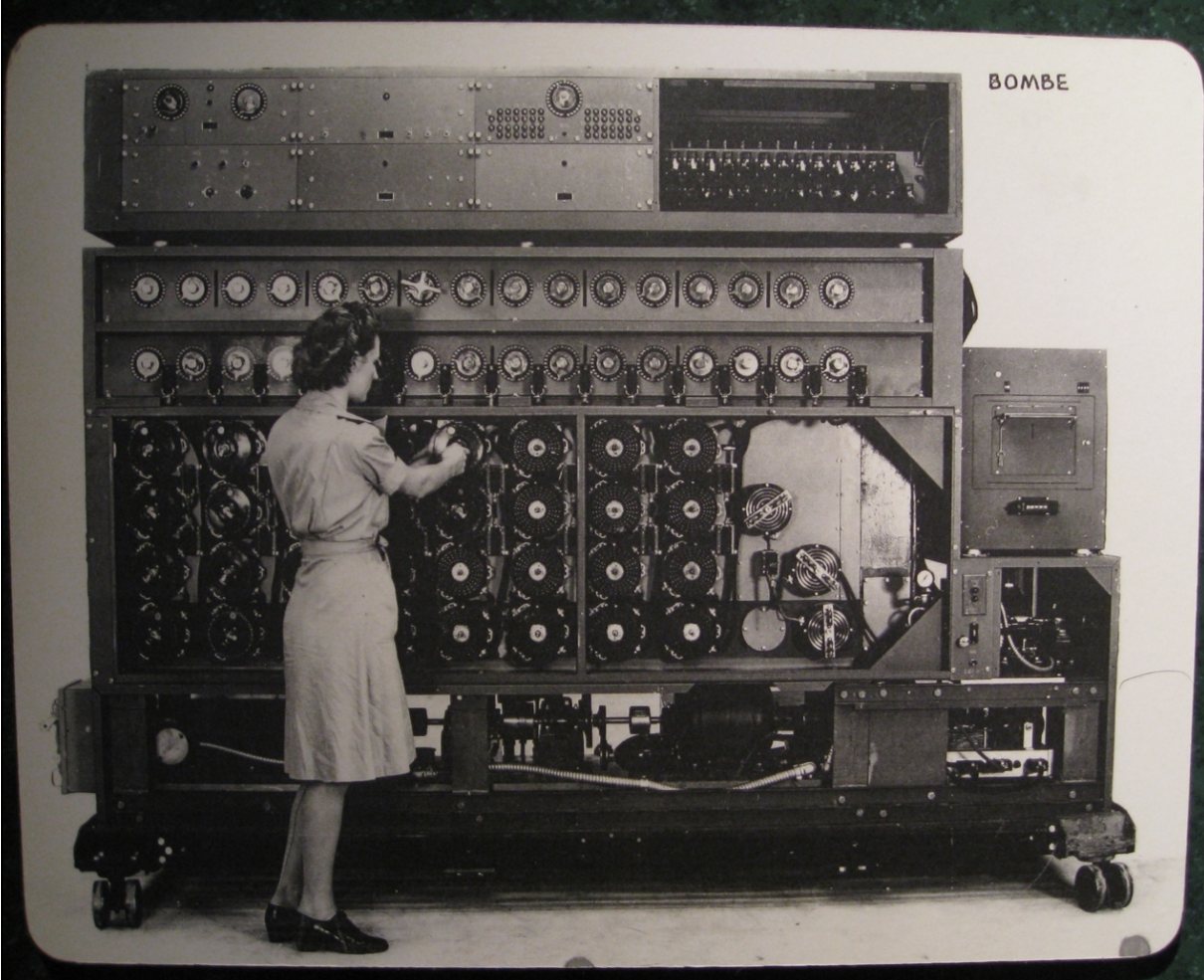




Chef

- But you already guessed that, didn't you?

Chef is Infrastructure as Code



- Programmatically provision and configure
- Treat like any other code base
- Reconstruct business from code repository, data backup, and bare metal resources.

<http://www.flickr.com/photos/louisb/4555295187/>



OPSCODE



Programs

- Chef generates configurations directly on nodes from their run list
- Reduce management complexity through abstraction
- Store the configuration of your programs in version control

<http://www.flickr.com/photos/ssoosay/5126146763/>

Declarative Interface to Resources

- Define Policy
- Say what, not how
- Pull not Push

<http://www.flickr.com/photos/bixentro/2591838509/>





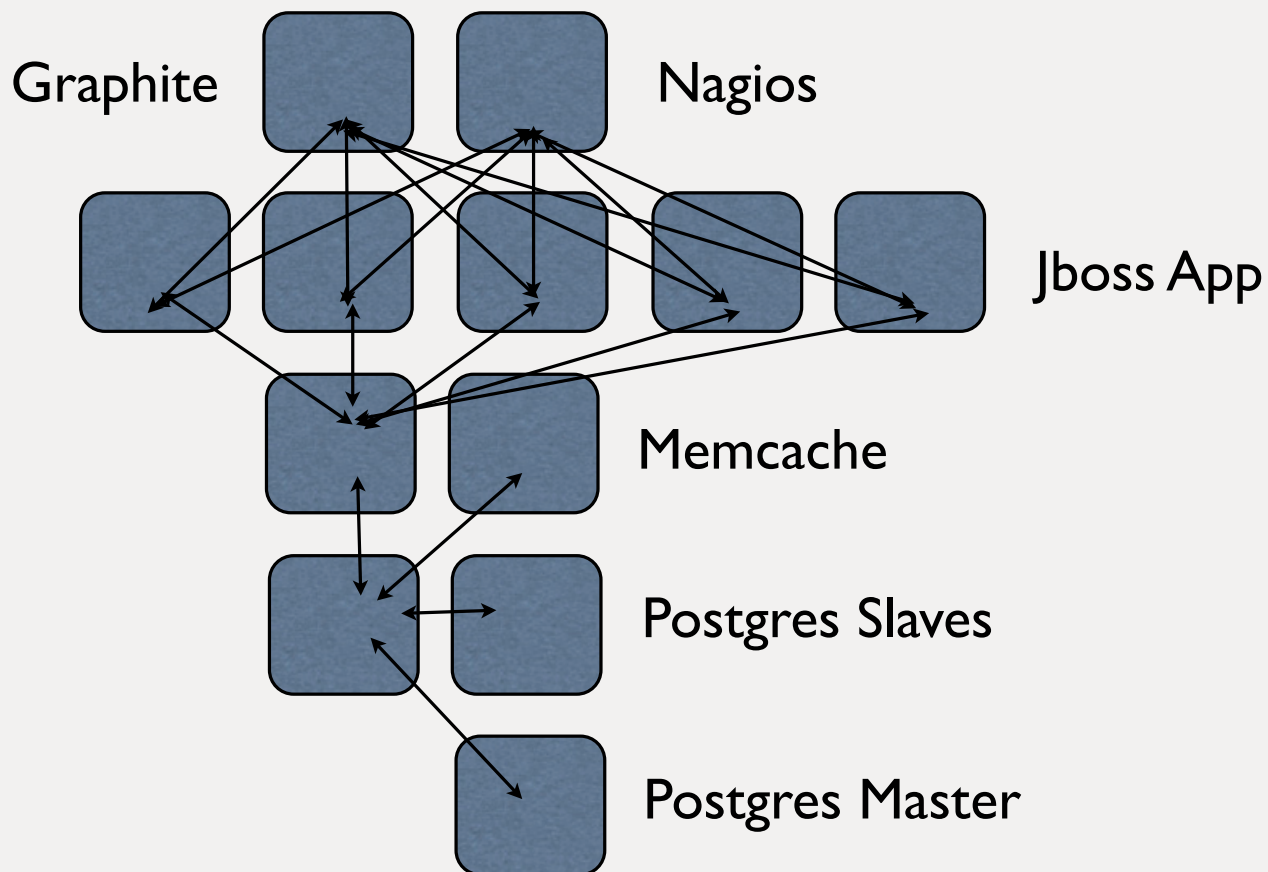
OPSCODE

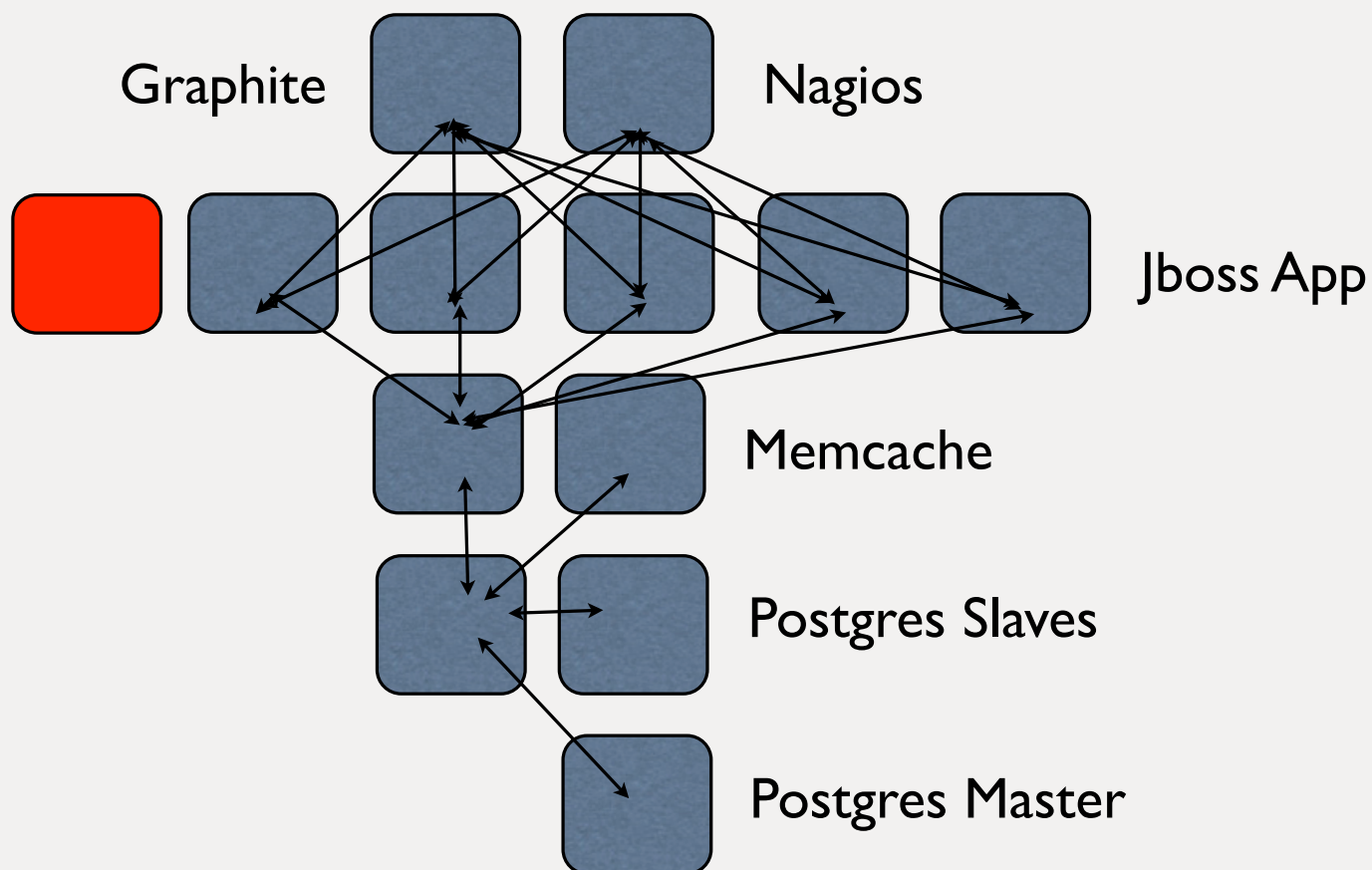
That looks like this

```
package "ntp" do
  action :install
end
```

```
template "/etc/ntp.conf" do
  source "ntp.conf.erb"
  owner "root"
  group "root"
  mode 0644
  action :create
  variables(:time_server => "time.example.com")
  notifies :restart, "service[ntpd]"
end
```

```
service "ntpd" do
  action [:enable, :start]
end
```

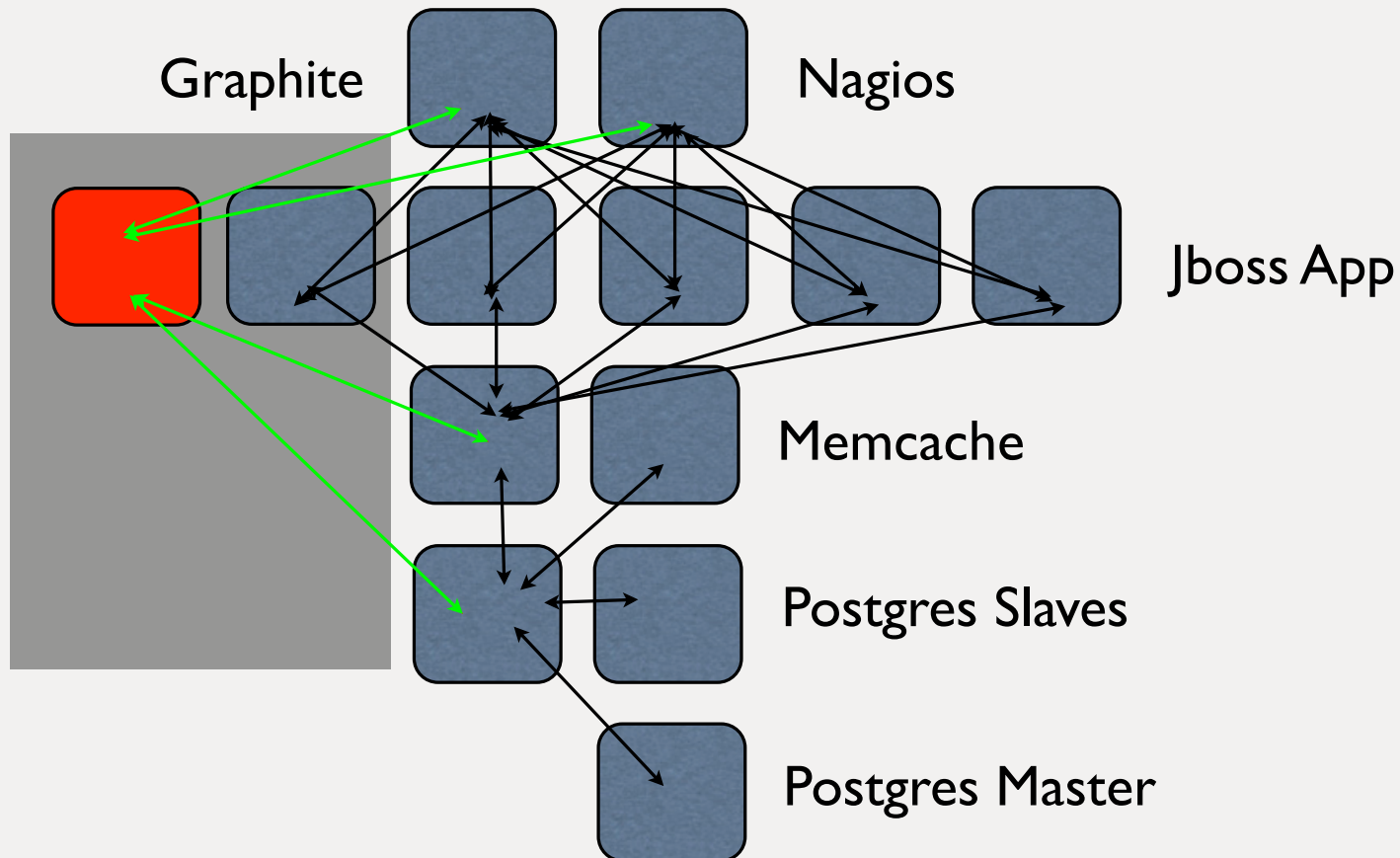






OPSCODE

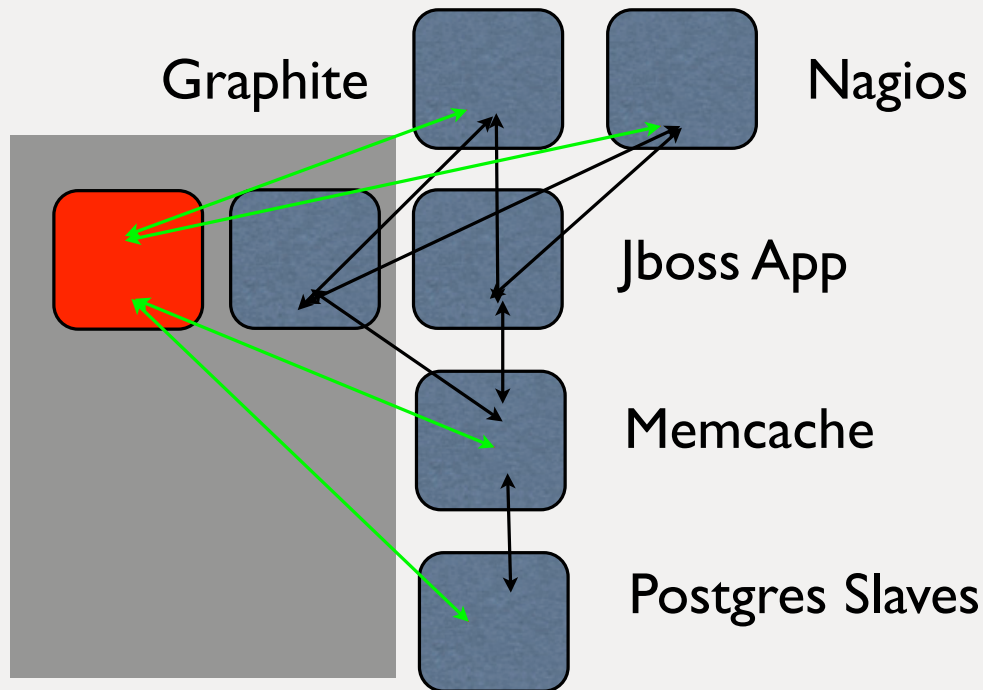
This can happen automatically





OPSCODE

Count the resources



- 12+ resource changes for 1 node addition

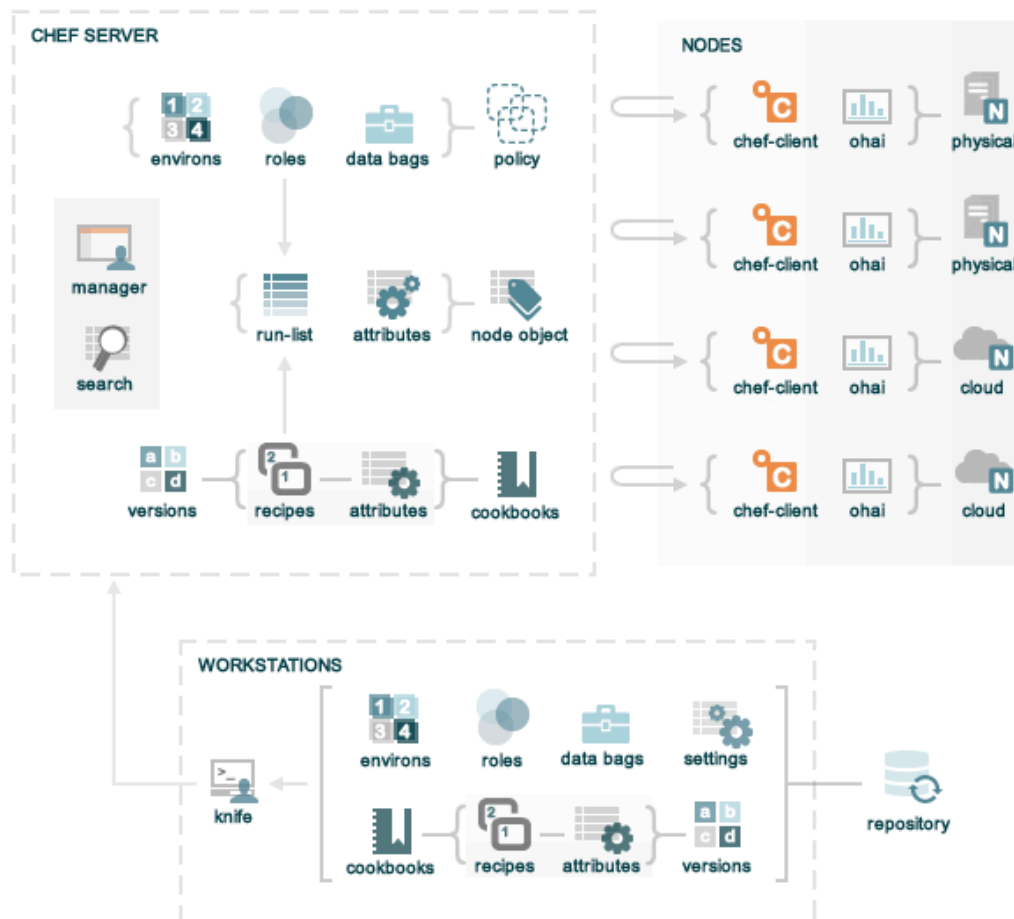
- Load balancer config
- Nagios host ping
- Nagios host ssh
- Nagios host HTTP
- Nagios host app health
- Graphite CPU
- Graphite Memory
- Graphite Disk
- Graphite SNMP
- Memcache firewall
- Postgres firewall
- Postgres authZ config

Getting Started



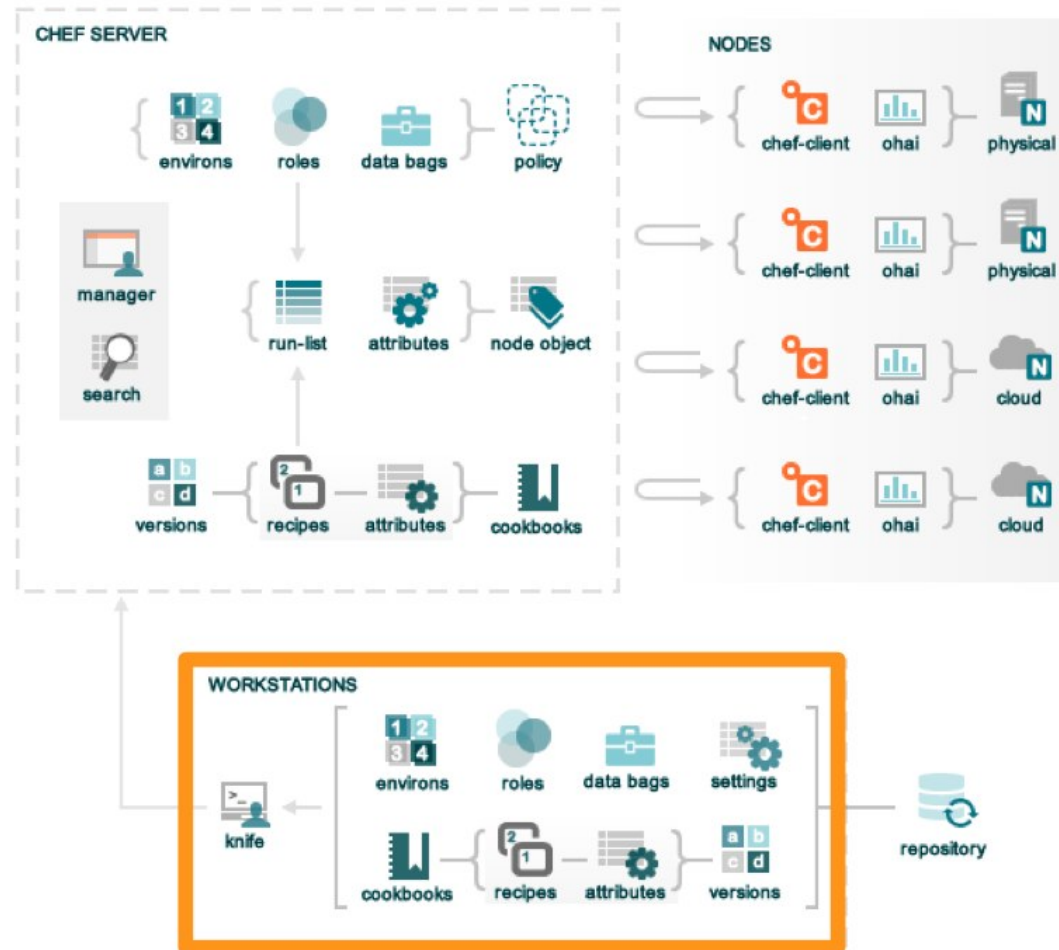
- Workstation Setup
- Chef Server Account
- Chef Repository
- Remote target managed node

Landscape of Chef-managed Infrastructure



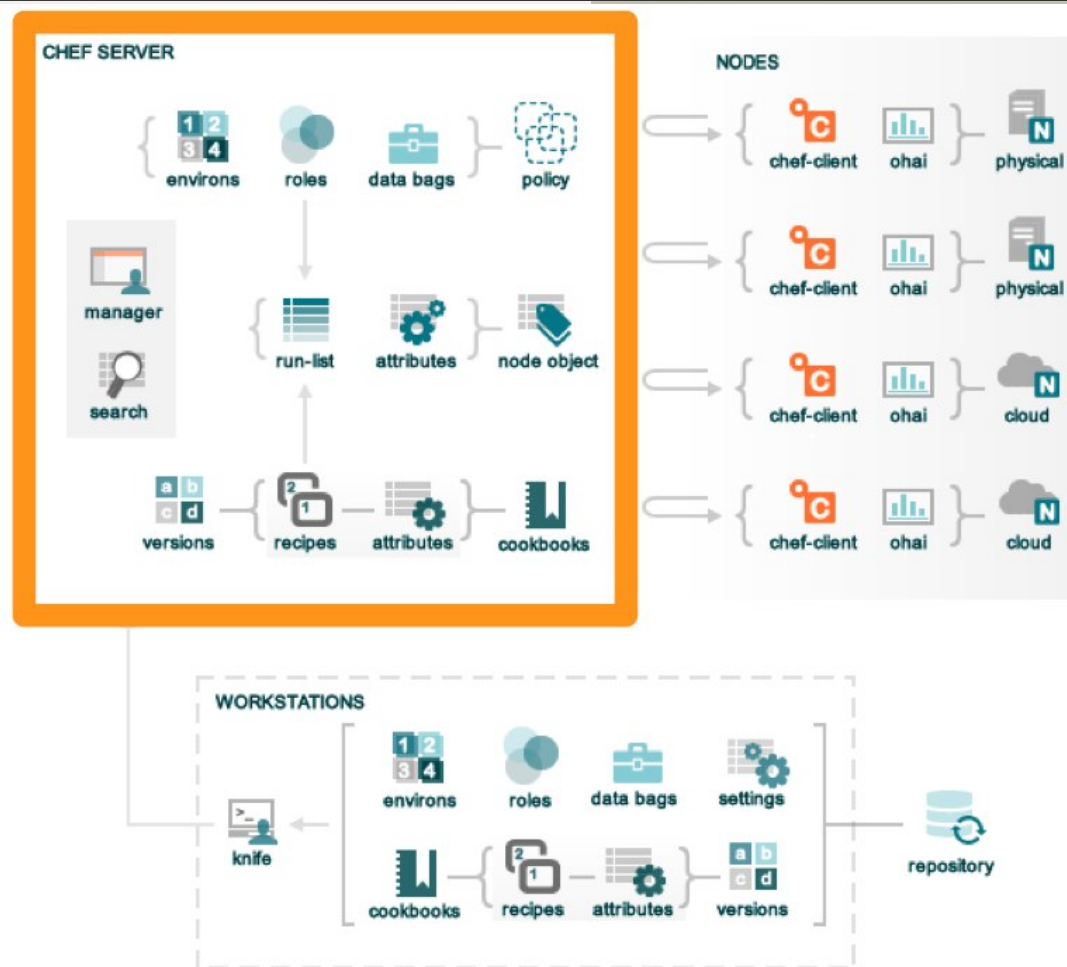
Workstation Setup

- Install Chef (if not already installed)
- <https://www.opscode.com/chef/install/>



Your Chef Server for this class...

- Set up Chef Server Account
- Opscode Hosted Chef
- <https://manage.opscode.com>





Sign-up for Hosted Chef

Hosted Chef

Instant access to a highly available, dynamically scalable, fully managed and supported automation environment

Opscode's Hosted Chef combines the freedom and flexibility of Chef with the reliability and speed of Opscode. No matter how complex the realities of your business, Hosted Chef makes it easy to deploy servers and scale applications throughout your entire infrastructure. Because it combines the fundamental elements of configuration management and service oriented architectures with the full power of Ruby, Hosted Chef makes it easy to create a fully automated infrastructure.

[Download the fact sheet](#)

Plans & Pricing

	Trial
Monthly Fees	\$0
Nodes	5
Users	5
Standard Support	Included
Onsite Training	Available

[Free Trial >](#)

About you

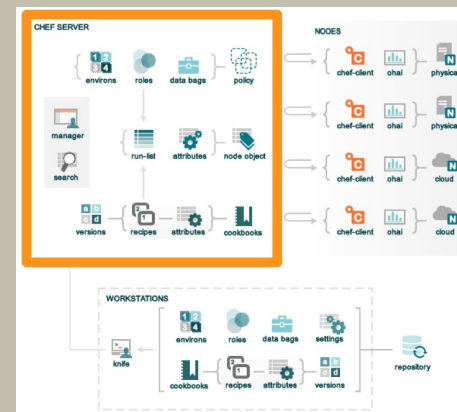
1 2 3

First Name *	Middle Name	Last Name *
Username *		Email Address *
Password *		Confirm Password *
Organization Name *		Organization Short Name *
Phone Number *		Company *
Country *		State *

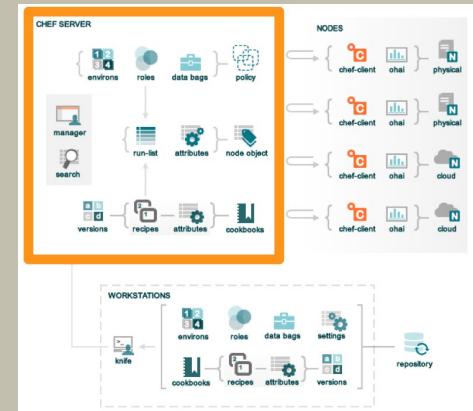
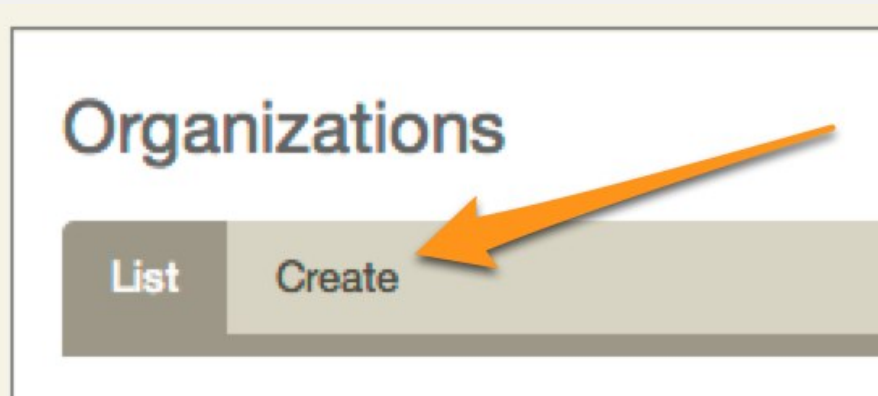
☐ I agree to the [Terms of Service](#), [Opscode Platform Customer Agreement](#), and [Opscode Service Level Agreement](#).

[< Back](#)

[Next >](#)



Create an Organization





Create New Organization

Create New Organization

List

Create

Organization Full Name (e.g. Opscode, Inc)

NH DC Training

Organization Short Name (e.g. opscience)

nhdctraining

Choose the organization's plan

Use the Free Plan

Premium
\$600/month

100 nodes
50 users
10 environments

Select

Standard
\$300/month

50 nodes
20 users
5 environments

Select

Launch
\$120/month

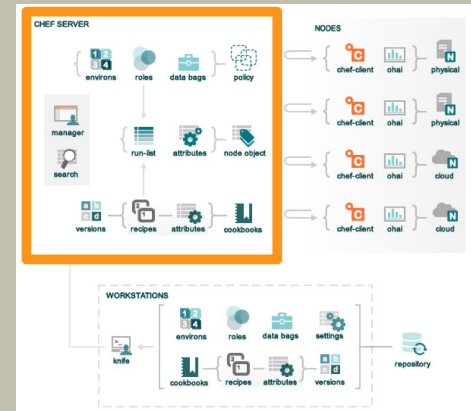
20 nodes
10 users
3 environments

Select

Free
\$0/month

5 nodes
2 users
1 environments

Select



Organization Short Name must be GLOBALLY unique!



Download the Validation Key and Knife Config

Organization: training

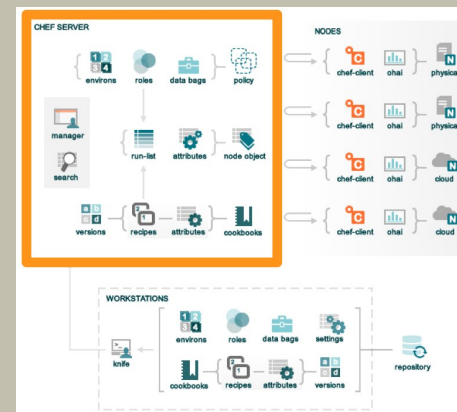
List Create

Save the validation key and Knife configuration file

This private key will be required when you interact with Opscode Platform APIs. Opscode does not keep a copy so please store it somewhere safe.

Knife is Chef's command-line tool. You can download a pre-configured copy of the configuration file.

[Download validation key](#) | [Generate knife config](#)



Logged in as: [nathenharvey](#) | [Organizations](#) | [Logout](#)

Account Management

[My Profile](#)

[Change Password](#)

[View/Edit Plan](#)

[Billing](#)

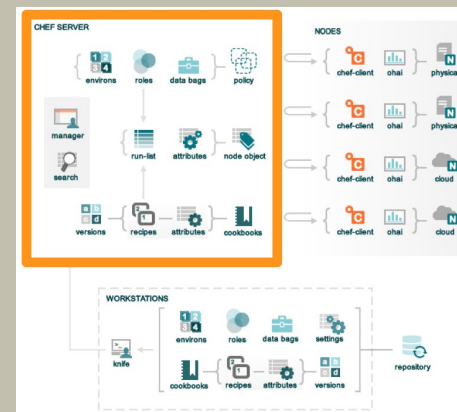


Nathen Harvey

Member for: over 2 years

To add or update your avatar image, please visit [Gravatar](#).

- Only if you don't have your user key with you today!



Reset User Key

If you've lost your private key, or would like to replace it, click the button below.

When you get a new key, **your old key will stop working.**

This private key **replaces your old key.**

We do not keep a copy so please store it somewhere safe.

[Get a new key](#)

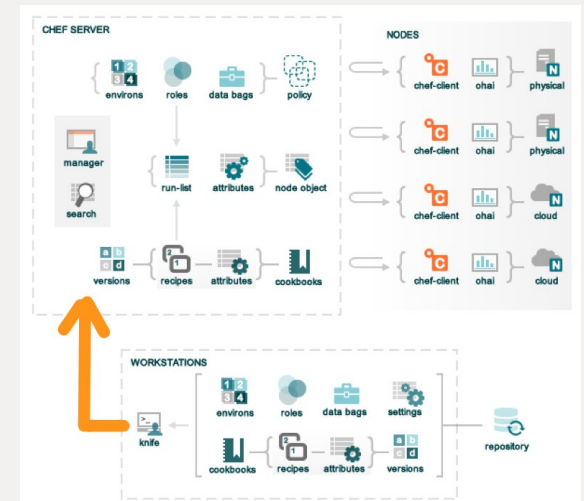
ec2-based Instance

- `ec2-STUDENT_ID.compute-1.amazonaws.com`
- Ubuntu 12.04
- SSH
 - *Username: **opscode***
 - *Password: **opscode***



```
# copy your user key, validation key and knife config:
> cp ~/Downloads/ORGNAME-validator.pem .chef
> cp ~/Downloads/USERNAME.pem .chef
> cp ~/Downloads/knife.rb .chef

> ls .chef
ORGNAME-validator.pem
USERNAME.pem
knife.rb
```



```
> knife --version
```

```
Chef: 11.4.0
```

Your version may differ, that's okay!

```
> knife client list
```

```
ORGNAME-validator
```



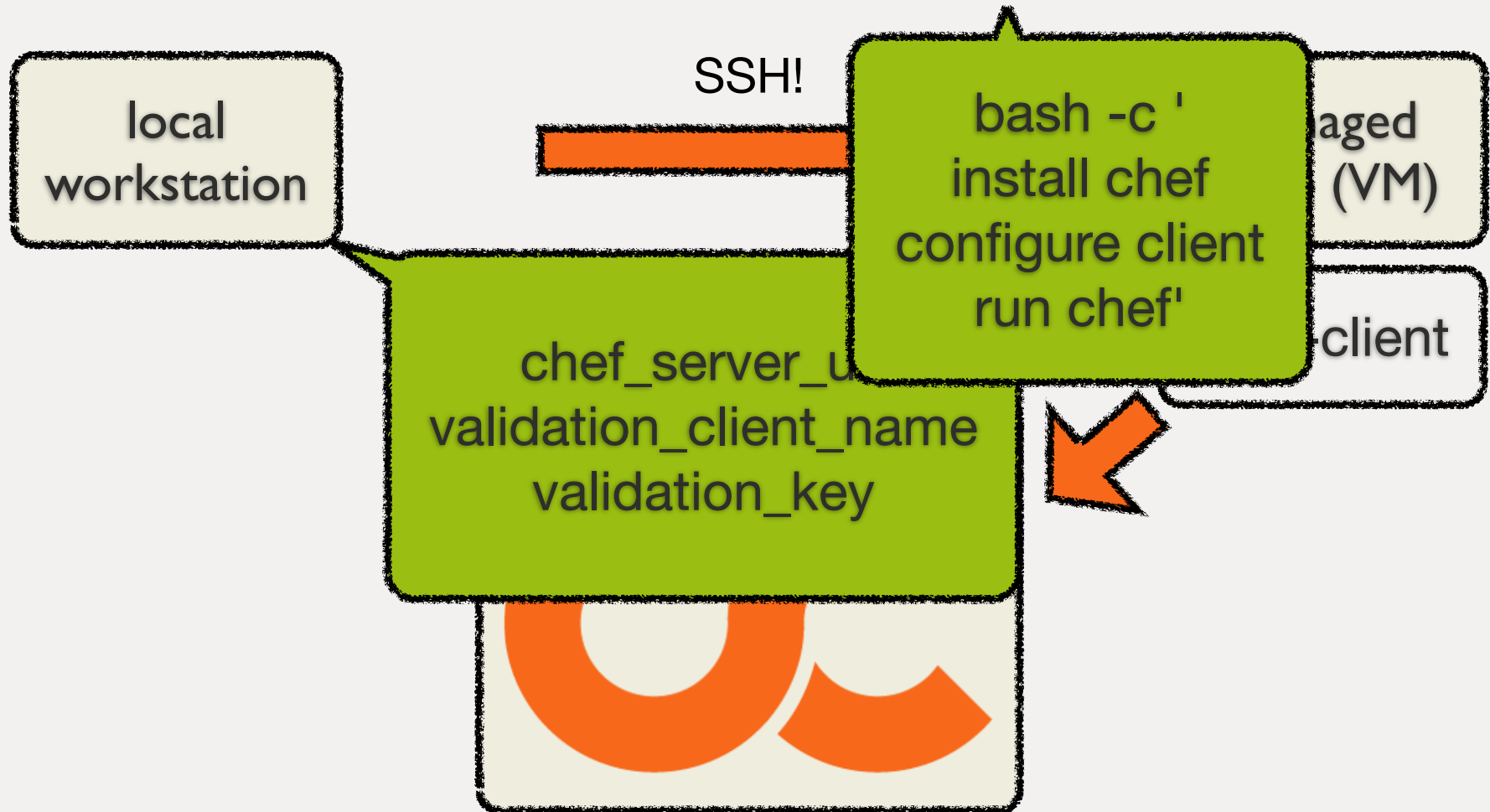
Bootstrap the Target Instance



```
> knife bootstrap IPADDRESS --sudo -x USERNAME -P PASSWORD
```

```
Bootstrapping Chef on target  
target Starting Chef Client, version 11.4.0
```

```
knife bootstrap IPADDRESS --sudo -x USERNAME -P PASSWORD
```



Chef 101 Terminology



***chef-client* runs on your
systems**

chef-client talks to a Chef
Server

API Clients authenticate with RSA keys

The server has the public key

**Configured, or managed
systems are called **Nodes****

Knife is the command-line
user's tool for Chef.

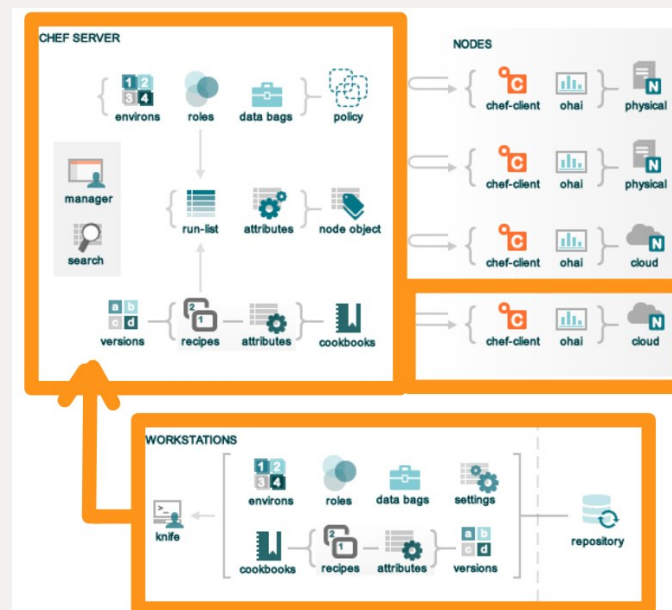
Current Status - Managed Node



```
> knife node list
target1
```

```
> knife client list
target1
ORGNAME-validator
```

```
> knife node show target1
Node Name:    target1
Environment:  _default
FQDN:         target1
IP:           10.12.13.201
Run List:
Roles:
Recipes:
Platform:     ubuntu 12.04
```



```
> knife node show --help
```

```
> knife help node
```

```
> ssh opscience@target
```

```
opscience@target1:~$ ls /etc/chef
```

```
client.pem  client.rb  first-boot.json  validation.pem
```

```
$ cat /etc/chef/client.rb
log_level          :auto
log_location       STDOUT
chef_server_url    "https://chef.local/organizations/ORGNAME"
validation_client_name "ORGNAME-validator"
# Using default node name (fqdn)
```

```
$ cat /etc/chef/first-boot.json  
{"run_list":[]}
```

```
$ chef-client -h | grep -i json  
-j JSON_ATTRIBS, Load attributes from a JSON file or URL  
--json-attributes
```

- Remember from the authentication cycle:
Chef Server requires keys to authenticate.
- `client.pem` - private key for API client
- `validation.pem` - private key for
ORGNAME-validator

Managing resolv.conf





- Describe **cookbooks**
- Create a new **cookbook**
- Describe a **recipe**
- Use the **cookbook_file** resource
- Upload a **cookbook** to the **Chef Server**
- Describe a **run list**
- Read the output of a **chef-client** run

The Problem and the Success Criteria

- **Problem:** Use OpenDNS servers for DNS resolution.
- **Success Criteria:** OpenDNS responds to our DNS queries.
- Yes, this is a trivial example but will give you an easy introduction to using Chef for something you've done before.



- Identify the **resources** required
- Create a **cookbook**
- Write a **recipe**
- Update the **run list**
- Converge the **node**

- Have a type.
- Have a name.
- Have parameters.
- Take action to put the resource in the declared state.
- Can send notifications to other resources.

```
package "haproxy" do
  action :install
end

template "/etc/haproxy/haproxy.cfg" do
  source "haproxy.cfg.erb"
  owner "root"
  group "root"
  mode 0644
  notifies :restart, "service[haproxy]"
end

service "haproxy" do
  supports :restart => true
  action [:enable, :start]
end
```



- Identify the **resources** required
 - `cookbook_file`
- Create a **cookbook**
- Write a **recipe**
- Update the **run list**
- Converge the **node**

- A cookbook is like a “package” for Chef recipes.
- It contains all the recipes, files, templates, libraries, etc. required to configure a portion of your infrastructure
- Typically they map 1:1 to a piece of software or functionality.

```
$ knife cookbook create resolv
```

```
** Creating cookbook resolv  
** Creating README for cookbook: resolv  
** Creating CHANGELOG for cookbook: resolv  
** Creating metadata for cookbook: resolv
```

```
$ ls -1 cookbooks/resolv
```

```
CHANGELOG.md  
README.md  
attributes  
definitions  
files  
libraries  
metadata.rb  
providers  
recipes  
resources  
templates
```

```
$ ls -1 cookbooks/resolv
```

```
CHANGELOG.md  
README.md  
attributes  
definitions  
files  
libraries  
metadata.rb  
providers  
recipes  
resources  
templates
```



OPSCODE

Required steps

- Identify the **resources** required
- Create a **cookbook**
 - `cookbooks/resolv`
- Write a **recipe**
- Update the **run list**
- Converge the **node**

RULE THE CLOUD

```
$ vim cookbooks/resolv/recipes/default.rb
```

```
#  
# Cookbook Name:: resolv  
# Recipe:: default  
#  
# Copyright 2013, YOUR_COMPANY_NAME  
#  
# All rights reserved - Do Not Redistribute  
#
```



- The “default.rb” recipe for a given cookbook is referred to by the name of the cookbook (*resolv*)
- If we added another recipe to this cookbook named “dhcp.rb”, we would refer to it as *resolv::dhcp*

Add the following to `cookbooks/resolv/recipes/default.rb`

```
cookbook_file "/etc/resolv.conf" do
  action :create
  owner  "root"
  group  "root"
  mode   "0644"
end
```

So the resource we just wrote...

- Is a **cookbook_file** resource
- Whose **name** is “*/etc/resolv.conf*”
- With an **install** action
- And ownership- and permission-related **properties**

```
cookbook_file "/etc/resolv.conf" do
  action :create
  owner  "root"
  group  "root"
  mode   "0644"
end
```

- Where's the content?

```
$ vim cookbooks/resolv/files/default/resolv.conf
```

```
# OpenDNS servers
```

```
nameserver 208.67.222.222
```

```
nameserver 208.67.220.220
```

- Chef allows you to select the most appropriate file (or template) within a cookbook according to the platform of the node it is being executed on
 - node name (foo.bar.com)
 - platform-version (redhat-6.2)
 - platform-major (redhat-6)
 - platform
 - default
- 98% of the time, you will just use **default**

- Identify the **resources** required
- Create a **cookbook**
- Write a **recipe**
 - `cookbooks/resolv/default.rb`
 - `cookbooks/resolv/files/default/resolv.conf`
- Update the **run list**
- Converge the **node**

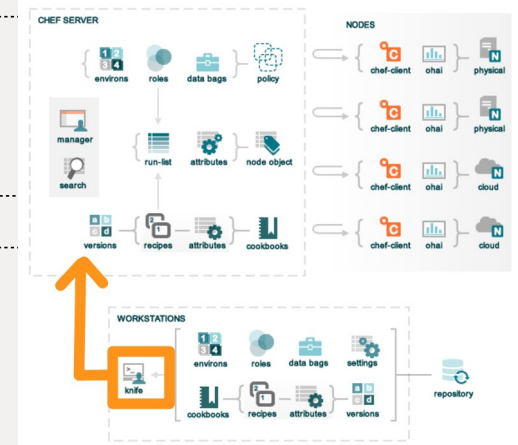


- Run lists specify what recipes the node should run, along with the order in which they should run

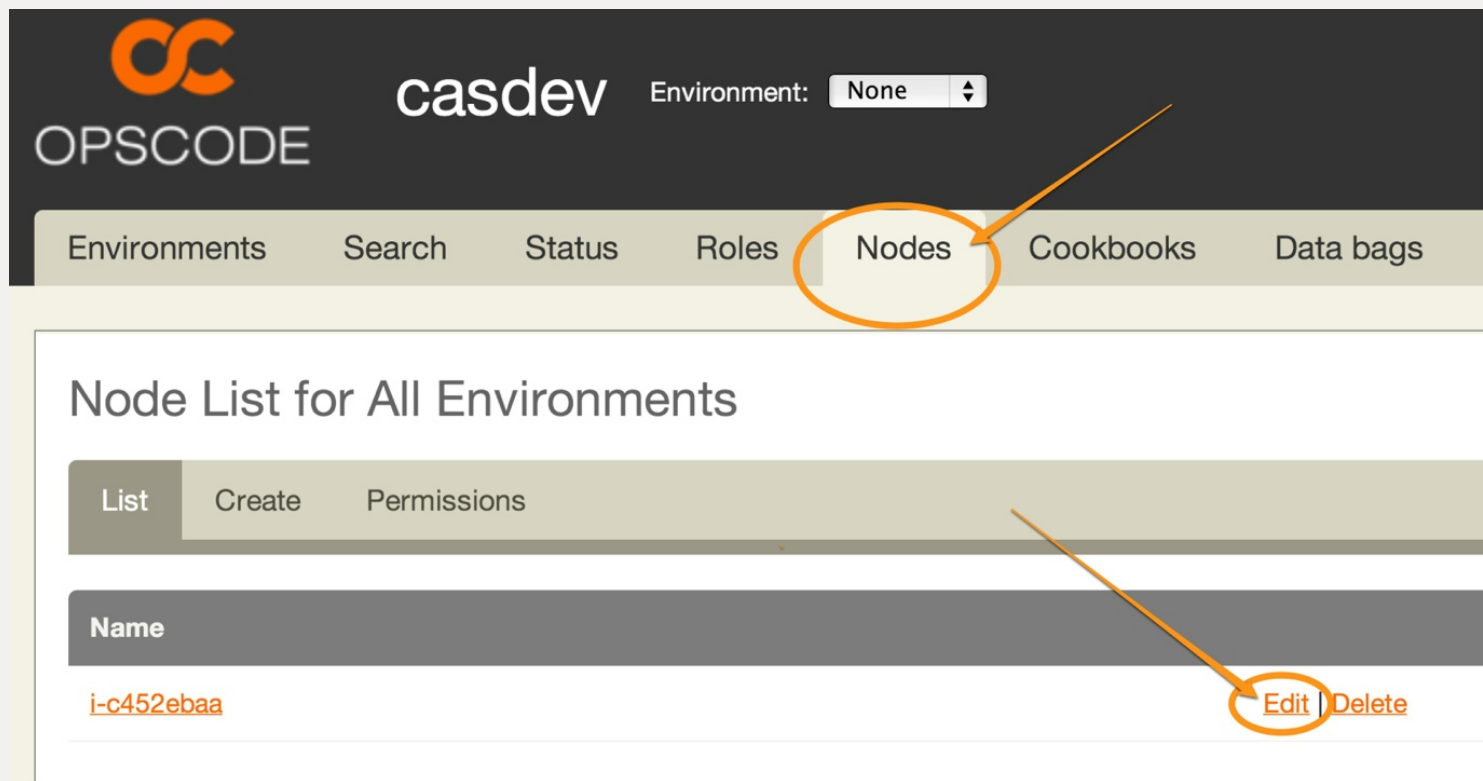
Exercise: Upload the resolv cookbook

```
$ knife cookbook upload resolv
```

```
Uploading resolv
Uploaded 1 cookbook.
```



[0.1.0]



The screenshot shows the Opscode Casdev interface. At the top, there's a dark header with the Opscode logo, the text 'casdev', and an 'Environment: None' dropdown. Below this is a navigation bar with tabs: 'Environments', 'Search', 'Status', 'Roles', 'Nodes', 'Cookbooks', and 'Data bags'. The 'Nodes' tab is selected and circled in orange, with an arrow pointing to it. Below the navigation bar, the main content area is titled 'Node List for All Environments'. It contains a table with columns 'List', 'Create', and 'Permissions'. The 'List' column has a sub-header 'Name'. Below this, there's a single row with the node name 'i-c452ebaa' and two buttons, 'Edit' and 'Delete', which are circled in orange with an arrow pointing to them.

List	Create	Permissions
Name		
i-c452ebaa		Edit Delete

Exercise: Add the resolv Cookbook to the Node's Run List



The screenshot shows the Opscode Chef interface. On the left, there are two panels: 'Available Roles' and 'Available Recipes'. The 'Available Recipes' panel has a search bar with the text 'resolv' entered. An orange arrow points from the search bar to the 'Run List' panel on the right. The text 'Drag & Drop' is written in orange next to the arrow.

```
$ knife node show NODENAME
```

```
Node Name:      i-c452ebaa
Environment:    _default
FQDN:           ip-10-151-72-210.ec2.internal
IP:             54.242.209.240
Run List:       recipe[resolve]
Roles:
Recipes:
Platform:      ubuntu 12.04
Tags:
```



OPSCODE

Required steps

- Identify the **resources** required
- Create a **cookbook**
- Write a **recipe**
- Update the **run list**
- Converge the **node**

RULE THE CLOUD

```
$ sudo chef-client
```

```
Starting Chef Client, version 11.4.0
resolving cookbooks for run list: ["resolver"]
Synchronizing Cookbooks:
  - resolver
Compiling Cookbooks...
Converging 1 resources
Recipe: resolver::default
  * cookbook_file[/etc/resolver.conf] action create
    - create a new cookbook_file /etc/resolver.conf
      --- /etc/resolver.conf      2013-03-16 04:37:45.255639757 +0000
      +++ /var/chef/cache/cookbooks/resolver/files/default/resolver.conf
2013-03-16 04:55:22.485063169 +0000
...
Chef Client finished, 1 resources updated
```

```
$ dig google.com +noall +stats
```

```
; <<>> DiG 9.8.1-P1 <<>> google.com +noall +stats  
;; global options: +cmd  
;; Query time: 4 msec  
;; SERVER: 208.67.222.222#53(208.67.222.222)  
;; WHEN: Sat Mar 16 05:00:15 2013  
;; MSG SIZE rcvd: 204
```

- You have just written your first Chef cookbook!
- (clap!)

```
Starting Chef Client, version 11.4.0
resolving cookbooks for run list: ["resolve"]
Synchronizing Cookbooks:
  - resolve
Compiling Cookbooks...
```

- We tell you the node's Run List

```
Converging 1 resources
Recipe: resolv::default
  * cookbook_file[/etc/resolv.conf] action create
    - create a new cookbook_file /etc/resolv.conf
      --- /etc/resolv.conf      2013-03-16 04:37:45.255639757 +0000
      +++ /var/chef/cache/cookbooks/resolv/files/default/resolv.conf  2013-03-16
04:55:22.485063169 +0000
      @@ -1,4 +1,3 @@
      -# Dynamic resolv.conf(5) file for glibc resolver(3) generated by
resolvconf(8)
      -#      DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
      -nameserver 172.16.0.23
      -search ec2.internal
      +# OpenDNS servers
      +nameserver 208.67.222.222
      +nameserver 208.67.220.220
```

```
Converging 1 resources
Recipe: resolv::default
 * cookbook_file[/etc/resolv.conf] action create
   - create a new cookbook_file /etc/resolv.conf
     --- /etc/resolv.conf      2013-03-16 04:37:45.255639757 +0000
     +++ /var/chef/cache/cookbooks/resolv/files/default/resolv.conf  2013-03-16 04:55:22.485063169
+0000
@@ -1,4 +1,3 @@
-# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
-#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
-nameserver 172.16.0.23
-search ec2.internal
+# OpenDNS servers
+nameserver 208.67.222.222
+nameserver 208.67.220.220
```

- We check to see if we need to create the resolv.conf file
- There is already one in place, whose contents are different than ours, so we back it up

- Actions on resources in Chef are designed to be **convergent**
- In practical terms, this means they only change the state of the system if they have to
- If a resource in Chef is properly configured, we move on to the next resource
- Convergent resources are **idempotent**
- Idempotent functions yield the same result with every application

Exercise: Re-run the Chef Client

```
$ sudo chef-client
```

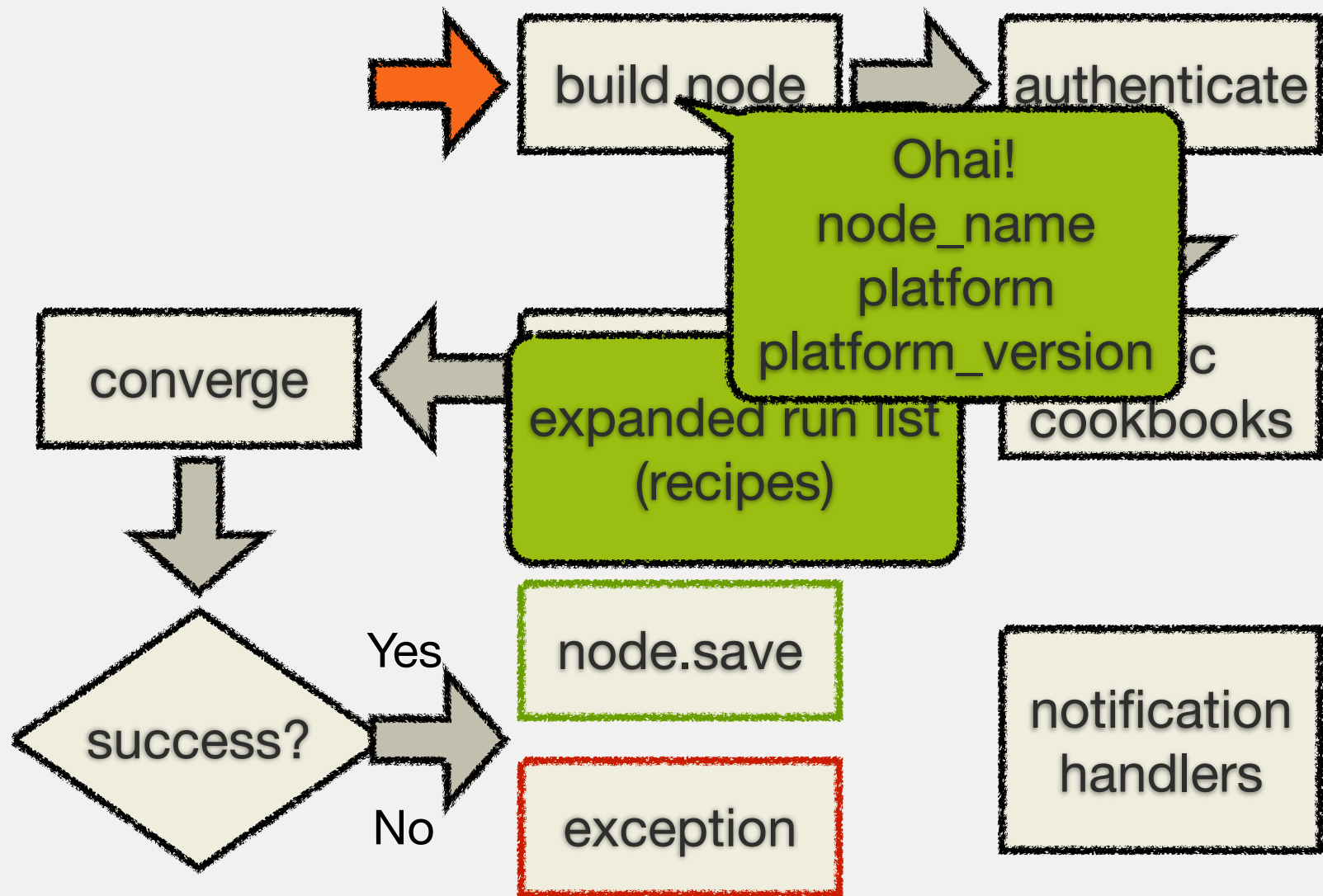
```
Starting Chef Client, version 11.4.0  
resolving cookbooks for run list: ["resolver"]  
Synchronizing Cookbooks:  
  - resolver  
Compiling Cookbooks...  
Converging 1 resources  
Recipe: resolver::default  
  * cookbook_file[/etc/resolver.conf] action create (up  
to date)  
Chef Client finished, 0 resources updated
```

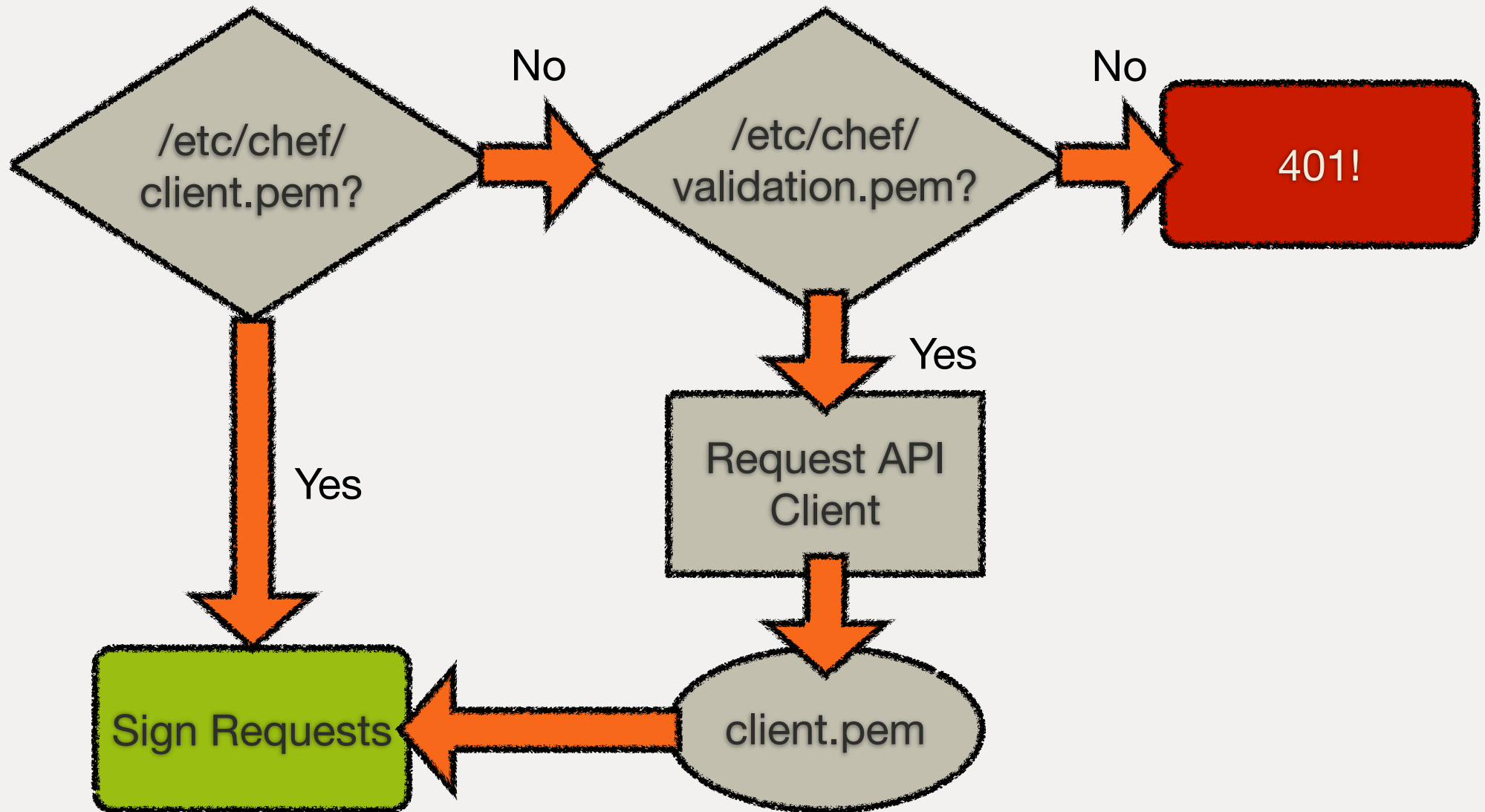


- What is a cookbook?
- How do you create a new cookbook?
- What is a recipe?
- What is a resource?
- How do you upload a cookbook to the Chef Server?
- What is a run list?

Anatomy of a Chef Run







Managing resolv.conf





OPSCODE

Objectives

- Describe **environments**
- Describe **attributes**
- Use the **template** resources
- Use **attributes**
- Describe **attribute** precedence

RULE THE CLOUD

The Problem and the Success Criteria

- **Problem:** Use OpenDNS servers for DNS resolution in development but Google servers for production.
- **Success Criteria:** OpenDNS responds to our DNS queries in development, Google in production
- Yes, this is a trivial example but will give you an easy introduction to using Chef for something you've done before.

- Create an **environment**
- Move the node to the production **environment**
- Update the **recipe**
- Converge the **node**



OPSCODE

Chef Environment

- An environment is a way to map an organization's real-life workflow to what can be configured and managed when using Chef Server.
- Ruby or JSON DSL

RULE THE CLOUD

```
$ mkdir environments  
$ vi environments/production.rb
```

```
name "production"  
description "Production Environment"
```

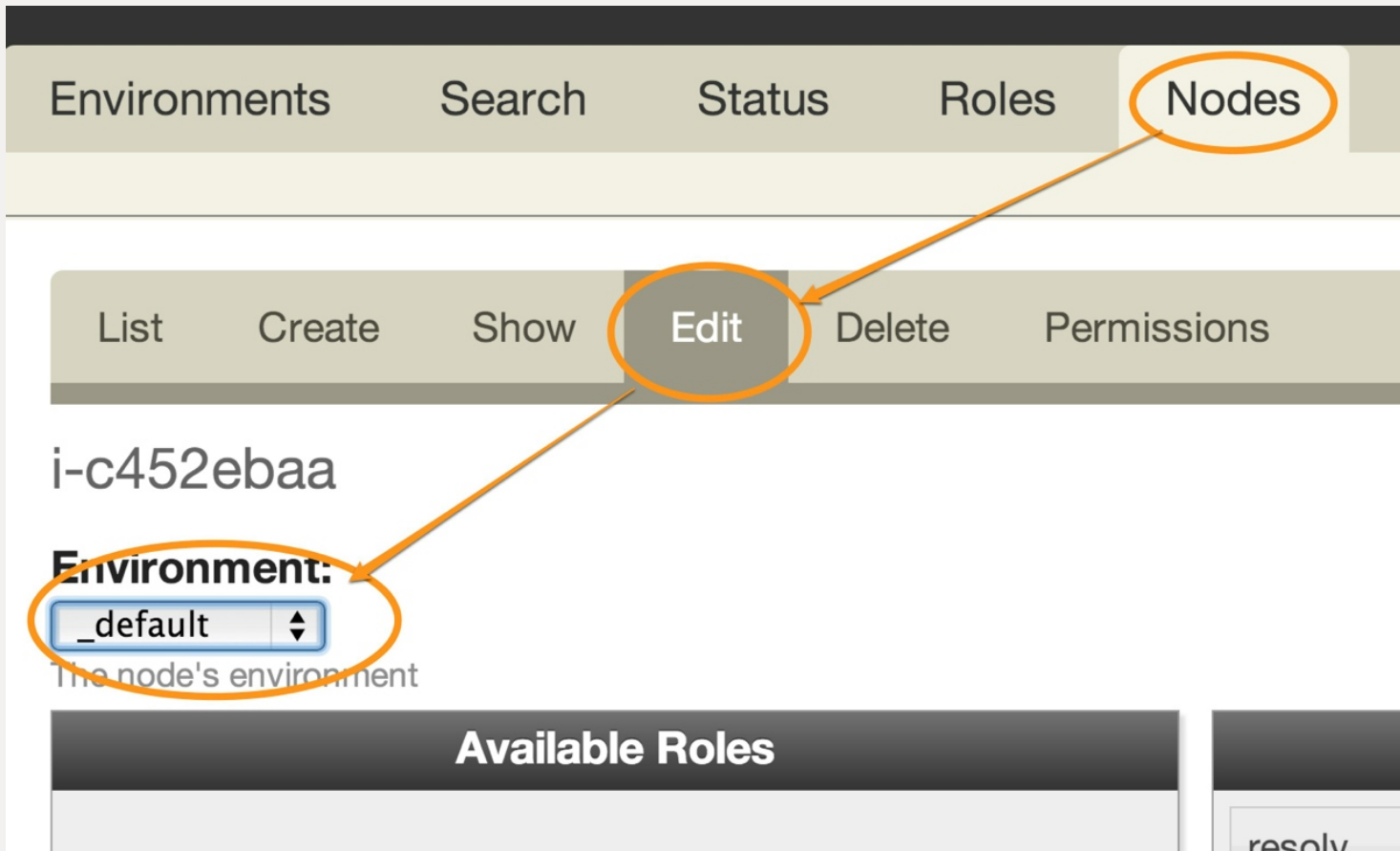
```
$ knife environment from file production.rb
```

Updated Environment production

```
$ knife environment show production
```

```
chef_type:                environment
cookbook_versions:
default_attributes:
description:              Production Environment
json_class:               Chef::Environment
name:                     production
override_attributes:
```

- Create an **environment**
 - `environments/production.rb`
- Move the node to the production **environment**
- Update the **recipe**
- Converge the **node**



The screenshot shows the Opscode Node management interface. The top navigation bar includes 'Environments', 'Search', 'Status', 'Roles', and 'Nodes'. The 'Nodes' tab is selected and circled in orange. Below this, a secondary navigation bar contains 'List', 'Create', 'Show', 'Edit', 'Delete', and 'Permissions'. The 'Edit' button is circled in orange, with an arrow pointing from the 'Nodes' tab to it. Below the navigation bars, the node ID 'i-c452ebaa' is displayed. Underneath, the 'Environment:' label is followed by a dropdown menu showing '_default', which is also circled in orange. An arrow points from the 'Edit' button to this dropdown. Below the dropdown is the text 'The node's environment'. At the bottom, there is a section titled 'Available Roles'.

Environments Search Status Roles **Nodes**

List Create Show **Edit** Delete Permissions

i-c452ebaa

Environment:

_default

The node's environment

Available Roles

resolv

```
$ knife node show NODENAME
```

```
Node Name:      i-c452ebaa
Environment:    production
FQDN:           ip-10-151-72-210
IP:             54.242.209.240
Run List:       recipe[resolve]
Roles:
Recipes:        resolv
Platform:       ubuntu 12.04
Tags:
```

- Create an **environment**
- Move the node to the production **environment**
- Update the **recipe**
- Converge the **node**

```
$ vim cookbooks/resolv/recipes/default.rb
```

```
template "/etc/resolv.conf" do
  action :create
  owner  "root"
  group  "root"
  mode   "0644"
end
```

```
$ vim cookbooks/resolv/templates/default/  
resolv.conf.erb
```

```
# This file managed by Chef!  
nameserver <%= node[ 'resolv' ][ 'nameserver' ] %>
```

```
$ vim environments/production.rb
```

```
name "production"  
description "Production Environment"  
default_attributes "resolv" => {  
  "nameserver" => "8.8.8.8"  
}
```

- Create an **environment**
- Move the node to the production **environment**
- Update the **recipe**
- Converge the **node**

```
$ sudo chef-client
```

```
Starting Chef Client, version 11.4.0
resolving cookbooks for run list: ["resolv"]
Synchronizing Cookbooks:
  - resolv
Compiling Cookbooks...
Converging 1 resources
Recipe: resolv::default
  * template[/etc/resolv.conf] action create
    - update template[/etc/resolv.conf] from e200db to 1912d2
      --- /etc/resolv.conf      2013-03-16 06:09:26.135447169 +0000
      +++ /tmp/chef-rendered-template20130316-3404-d6agp      2013-03-16
06:10:30.013804159 +0000
      @@ -1,3 +1,2 @@
      -# OpenDNS servers
      -nameserver 208.67.222.222
      -nameserver 208.67.220.220
      +# This file managed by Chef!
      +nameserver 8.8.8.4
```

```
$ dig google.com +noall +stats
```

```
; <<>> DiG 9.8.1-P1 <<>> google.com +noall +stats  
;; global options: +cmd  
;; Query time: 4 msec  
;; SERVER: 8.8.8.8#53(8.8.8.8)  
;; WHEN: Sat Mar 16 06:13:08 2013  
;; MSG SIZE rcvd: 204
```



- What is an environment?
- What languages can you use to define environments?
- What is an attribute?
- What are some places to specify attribute values?

Review





- Extremely simple example to illustrate some key concepts
- Manage configuration
- Respond to changes in requirements
- Build data-driven cookbooks

Questions?



Further Resources



- <http://opscode.com/>
- <http://learnchef.com>
- <http://community.opscode.com/>
- <http://docs.opscode.com>
- <http://wiki.opscode.com/>
- <http://lists.opscode.com>
- <http://youtube.com/user/Opscode>



- <http://foodfightshow.org>
- The Podcast Where DevOps Chef Do Battle
- Regular updates about new Cookbooks, Knife-plugins, and more
- Best Practices for working with Chef

#ChefConf
2013

Your Road to Automation

April 24 - 26, 2013
San Francisco, California

Register Early and Save

When you combine precedence and merge order, you get the complete picture of node attribute setting

	Attribute Files	Node/Recipe	Environment	Role
Default	1	2	3	4
Force Default	5	6		
Normal	7	8		
Override	9	10	12	11
Force Override	13	14		
Automatic	15			